

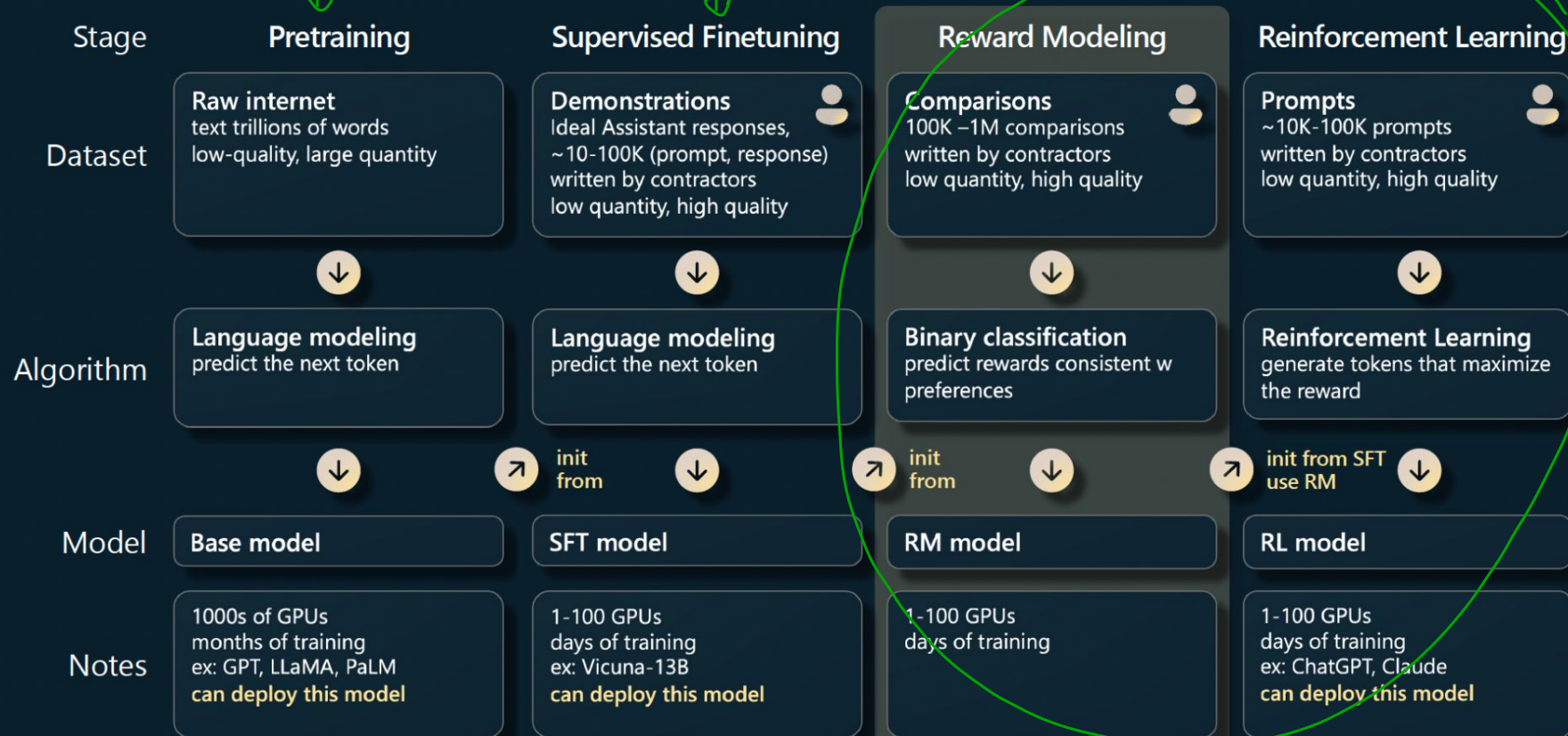
Introduction to Transformers

CS60216: Safety Fundamentals for Generative AI

Jan 5th, 2026

Our Goal:

GPT Assistant training pipeline



Let's start with the basics: Transformers

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com



Transformer Encoder-Decoder

- ❖ Transformer is introduced as an **encoder-decoder architecture**; later we will see **encoder-only** & **decoder-only** transformers
- ❖ **Encoder** produces a sophisticated representation of the source sequence that the decoder will use to condition its generation process
- ❖ **Decoder** generates one token at the time to produce a target sequence; it produces representations that combine the history and a new token

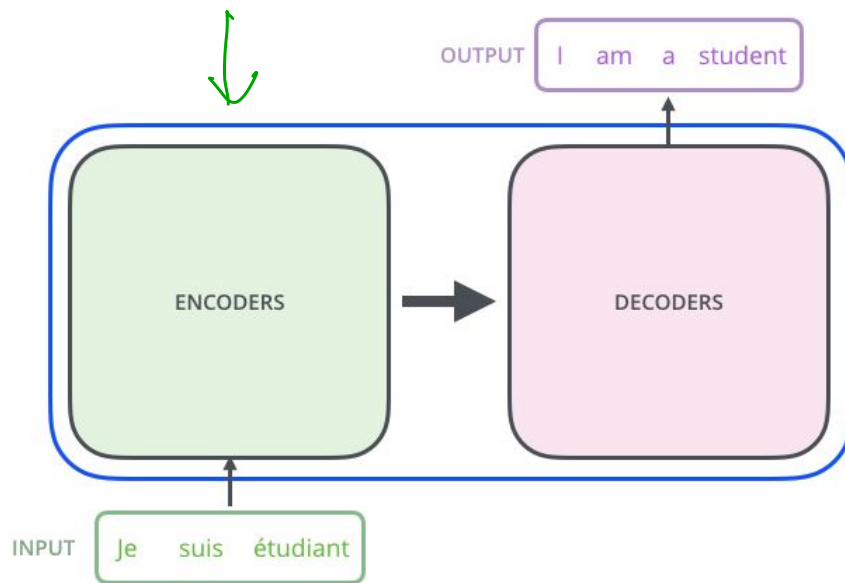
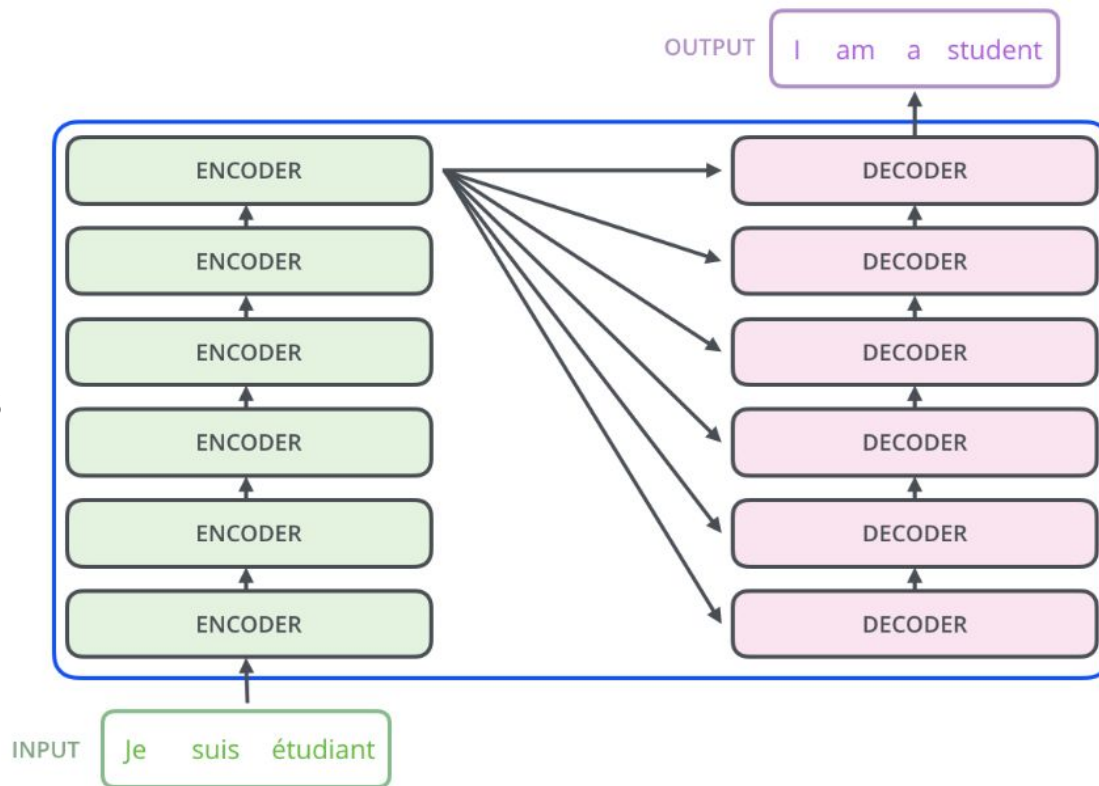


Figure: [Jay Alammar](#)

A stack of
encoder blocks



A stack of
decoder blocks

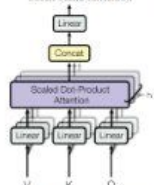
Figure: [Jay Alammar](#)

Attention mechanism

Scaled Dot-Product Attention



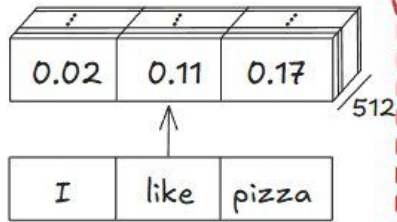
Multi-Head Attention



This is where the magic happens

Importance of different words are learnt relative to itself

So you can answer questions like
What do I like?
PIZZA



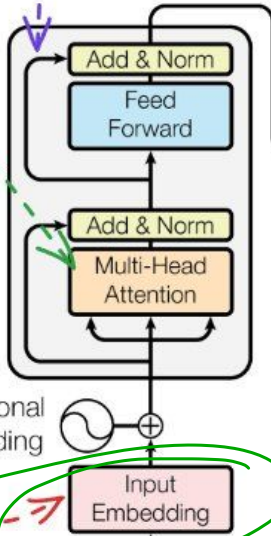
Converts each word* to an embedding of given size

*token will be a more appropriate term but more on that later

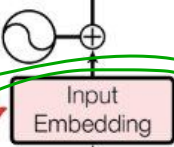
Residual connections

Make it easier for the network to preserve important information from earlier layers

Nx



Positional Encoding

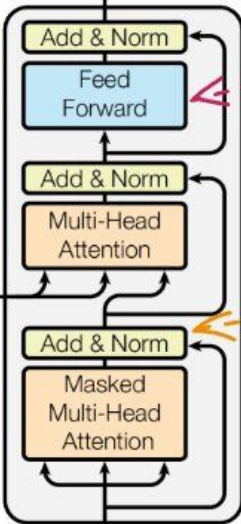


Inputs

Output Probabilities

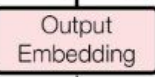
Softmax

Linear



Nx

Positional Encoding



Outputs (shifted right)

Linear layer $\rightarrow y = xA^T + b$

ReLU $\rightarrow \max(0, x)$

A network with sequence of linear layer, dropout and ReLU activation

$$y = \frac{x - E[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

Layer Norm to stabilize training

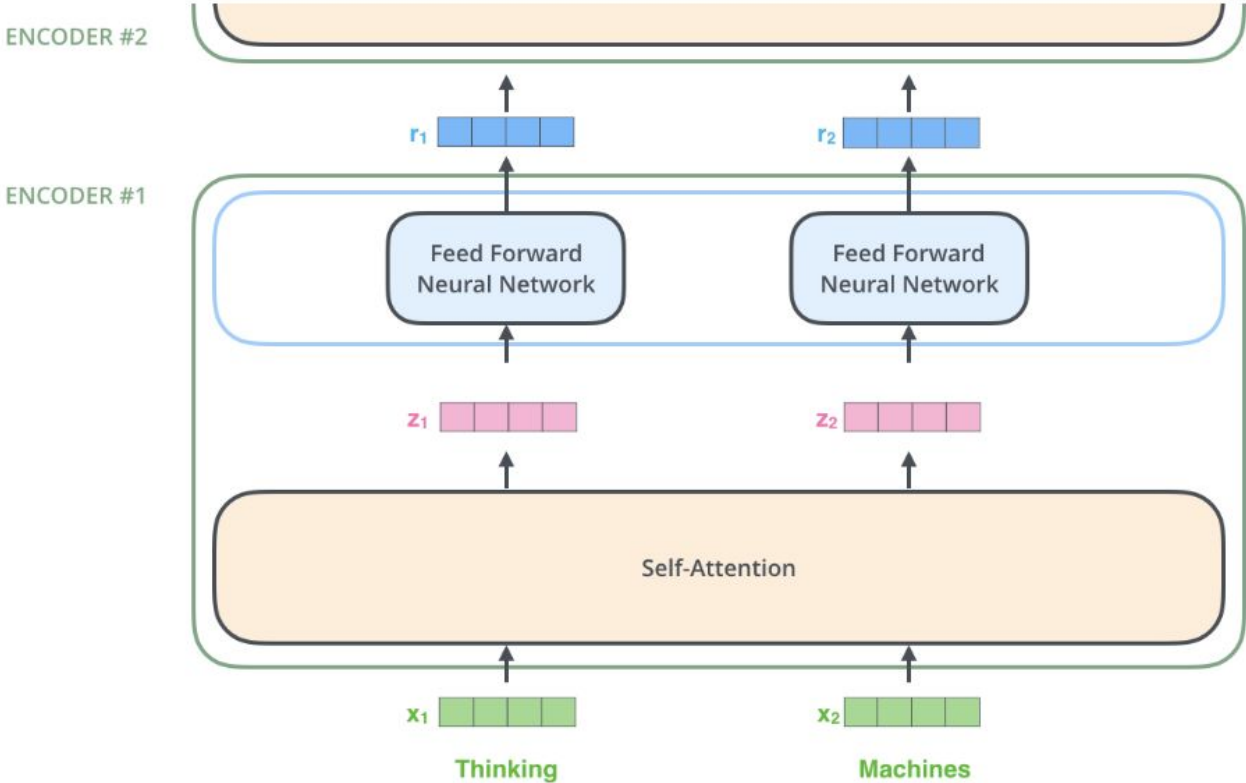
1st 2nd 3rd

I	like	pizza
---	------	-------

Adds positional information to the embeddings

This is required as the transformer processes requests in parallel

Each encoder block consists of **self-attention** & **FFNN**



Deeper layers get outputs of the previous layers as inputs

Input to each encoder block has the same size as original token embeddings

Figure: [Jay Alammar](#)

In the first layer, inputs are static token emeddings

Intuition for attention

The chicken didn't cross the road because it

What should be the properties of "it"?

The chicken didn't cross the road because it was too **tired**

The chicken didn't cross the road because it was too **wide**

At this point in the sentence, it's probably referring to either the animal or the street

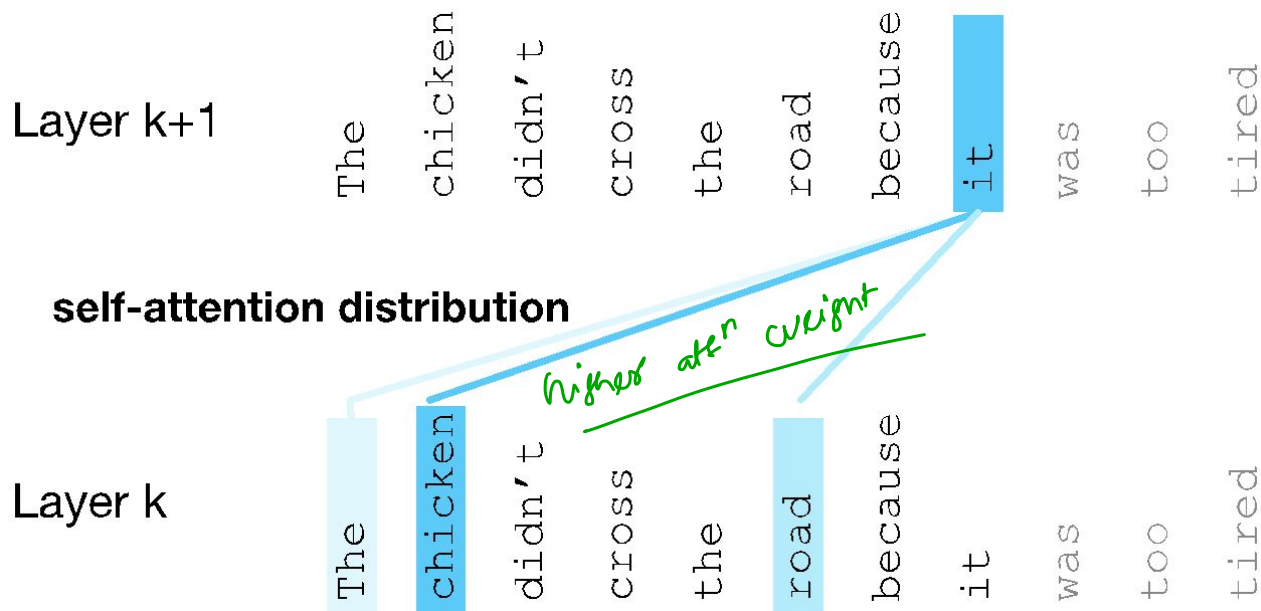
Intuition of attention

Build up the contextual embedding from a word by selectively integrating information from all the neighboring words

We say that a word "attends to" some neighboring words more than others

Intuition of attention

columns corresponding to input tokens

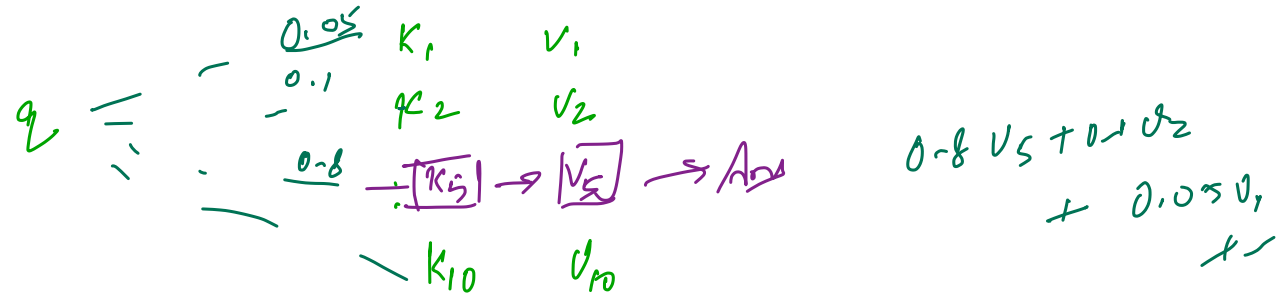


Intuition for Self-attention

Attention is based on key/value/query concept -- analogous to retrieval systems.

When you search for videos on Youtube

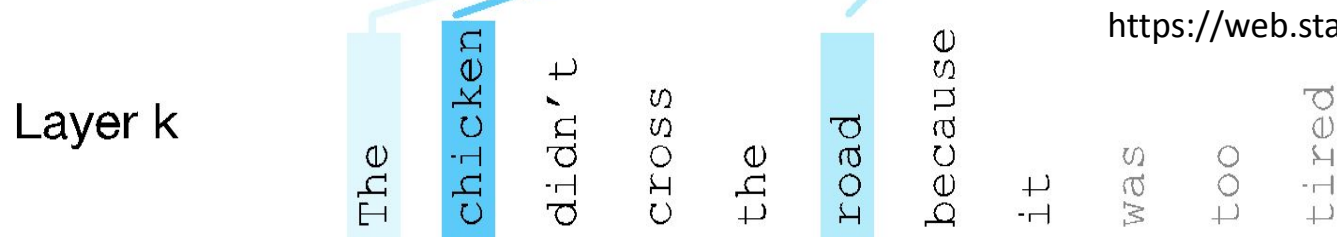
- The search engine will map your query (text in the search bar) against a set of keys (video title, description, etc.) associated with candidate videos in their database
- It will then present you the best matched videos (values).



Intuition of attention:

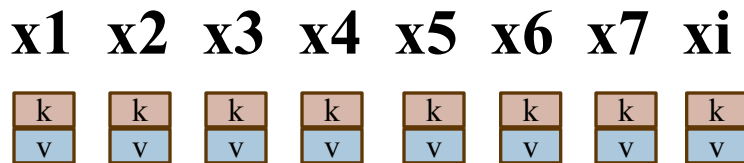


self-attention distribution



<https://web.stanford.edu/~jurafsky/slp3/>

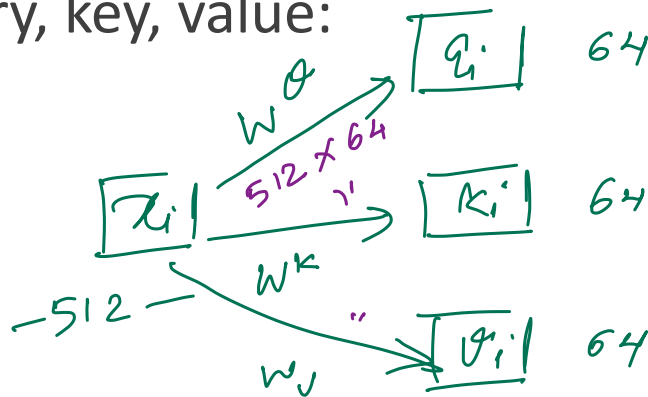
keys
values



An Actual Attention Head: slightly more complicated

We'll use matrices to project each vector $\mathbf{x}_i$ into a representation of its role as query, key, value:

- query: $\mathbf{W}^Q$
- key: $\mathbf{W}^K$
- value: $\mathbf{W}^V$



$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

An Actual Attention Head: slightly more complicated

Given these 3 representation of $\mathbf{x}_i$

To compute similarity of current element $\mathbf{x}_i$ with some prior element $\mathbf{x}_j$

We'll use dot product between $\mathbf{q}_i$ and $\mathbf{k}_j$.

And instead of summing up $\mathbf{x}_j$, we'll sum up $\mathbf{v}_j$

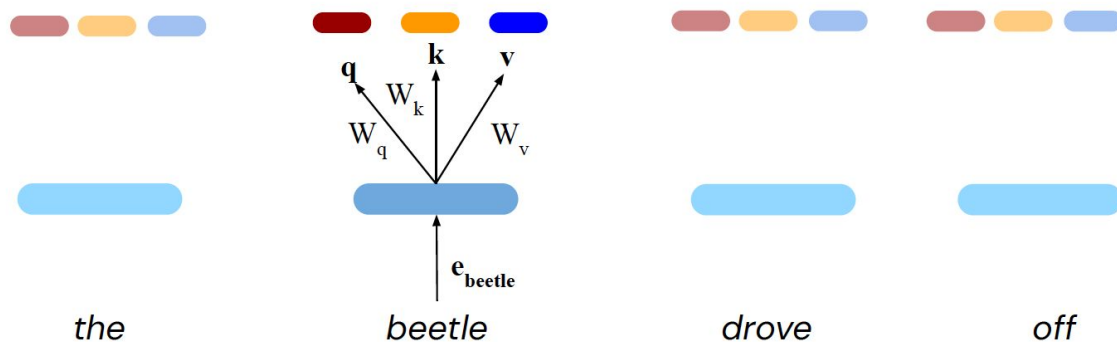
$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

Transformers: Self-attention over input

$$\mathbf{q} = \mathbf{e}_{beetle} W_q$$

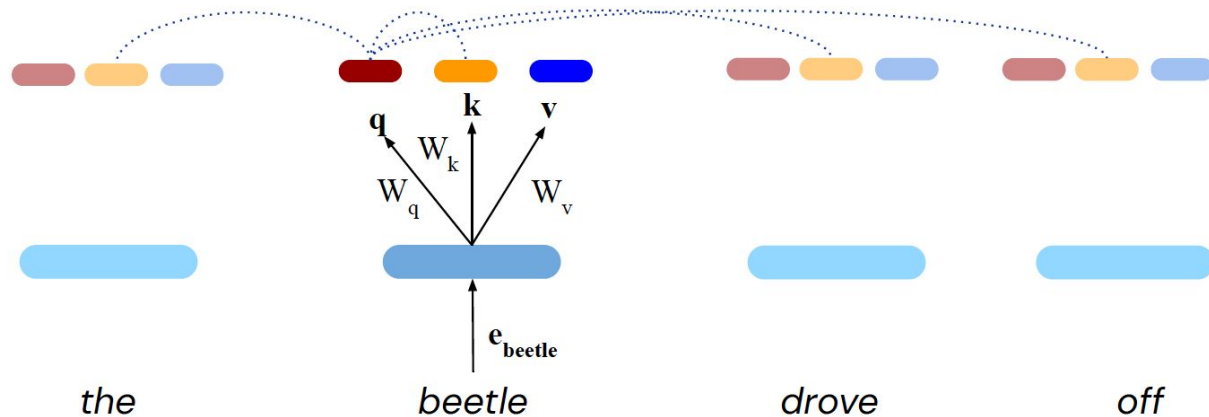
$$\mathbf{k} = \mathbf{e}_{beetle} W_k$$

$$\mathbf{v} = \mathbf{e}_{beetle} W_v$$



Source: <https://deepmind.com/learning-resources/deep-learning-lecture-series-2020>

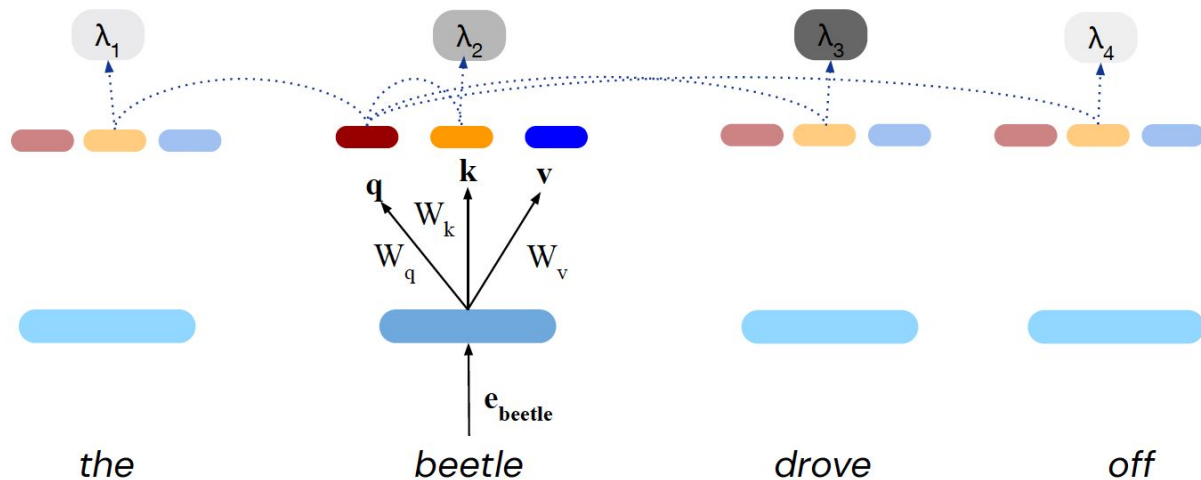
Self-attention over input embeddings



Source: <https://deepmind.com/learning-resources/deep-learning-lecture-series-2020>

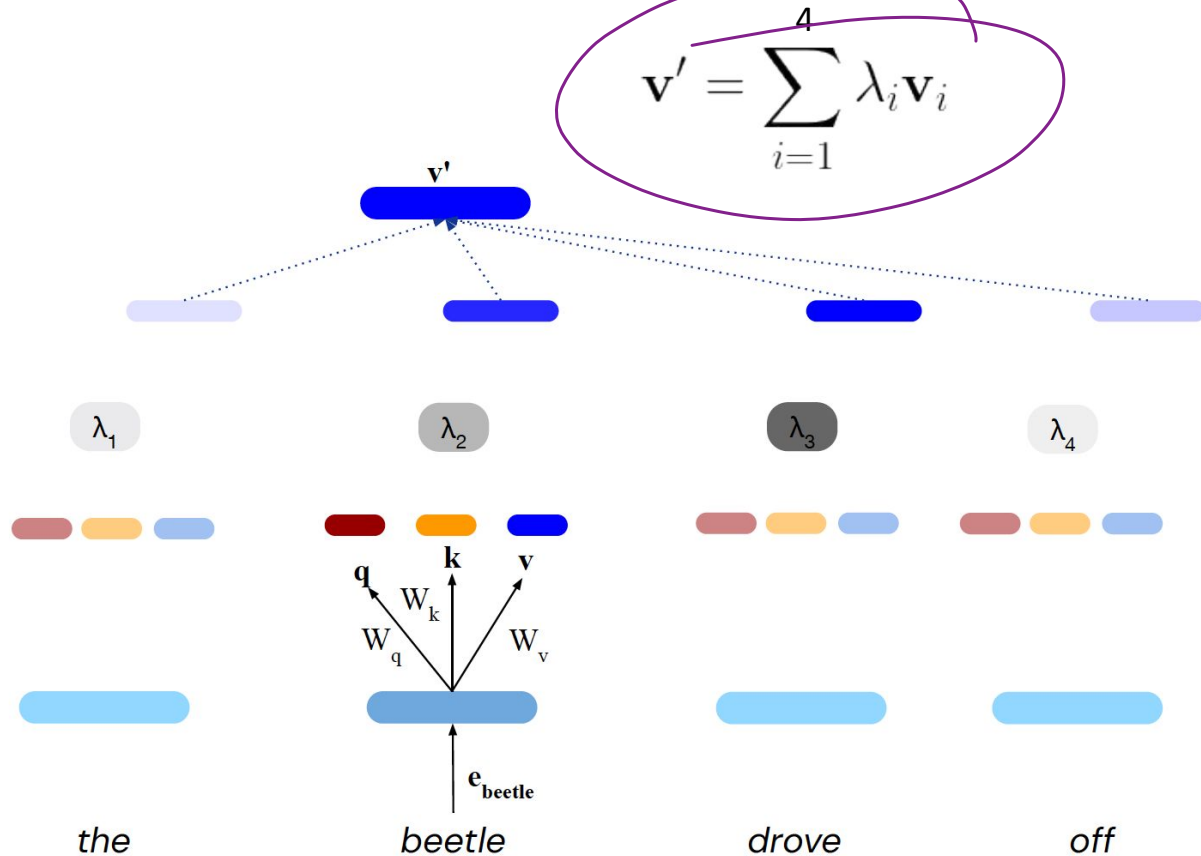
Self-attention over input embeddings

$$\lambda_i = \frac{e^{\mathbf{q} \cdot \mathbf{k}_i}}{\sum_{i=1}^4 e^{\mathbf{q} \cdot \mathbf{k}_i}}$$

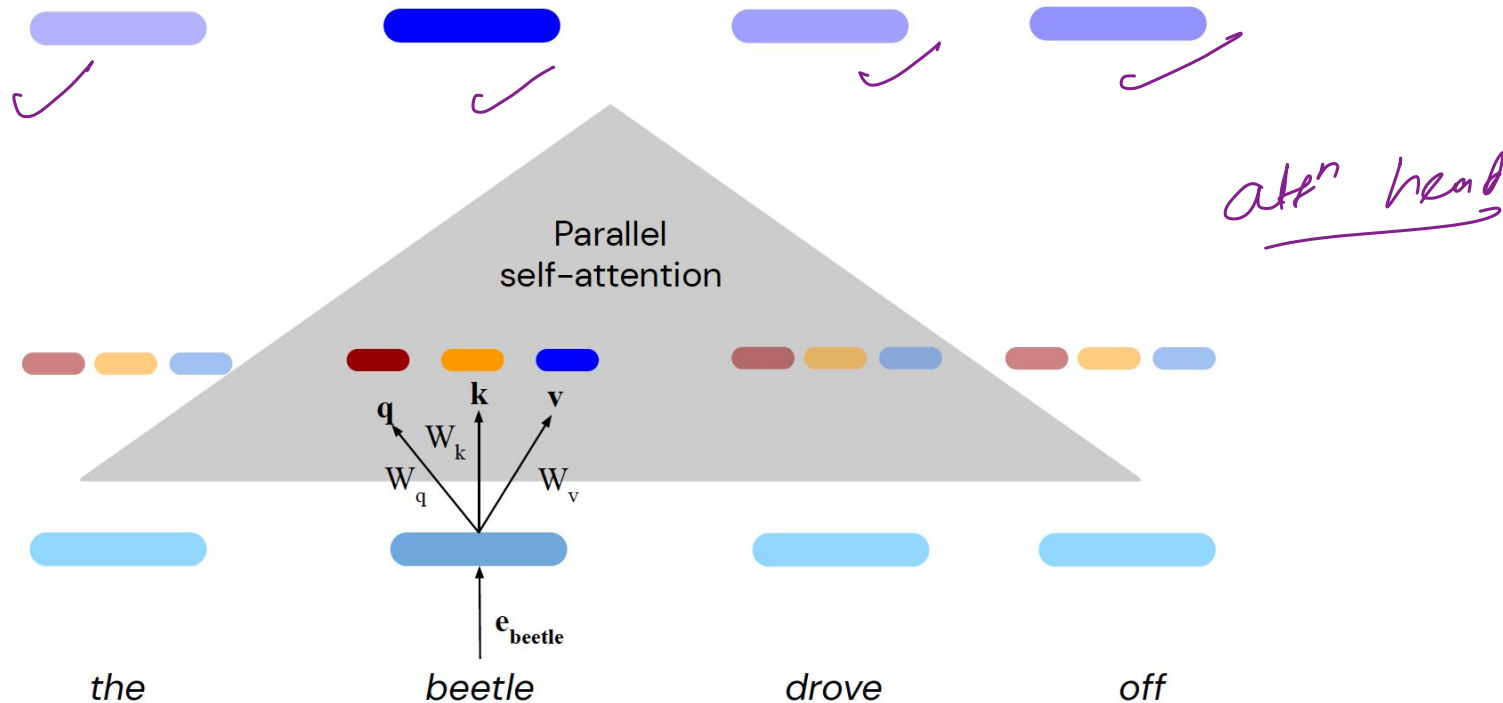


Source: <https://deepmind.com/learning-resources/deep-learning-lecture-series-2020>

Self-attention over input embeddings

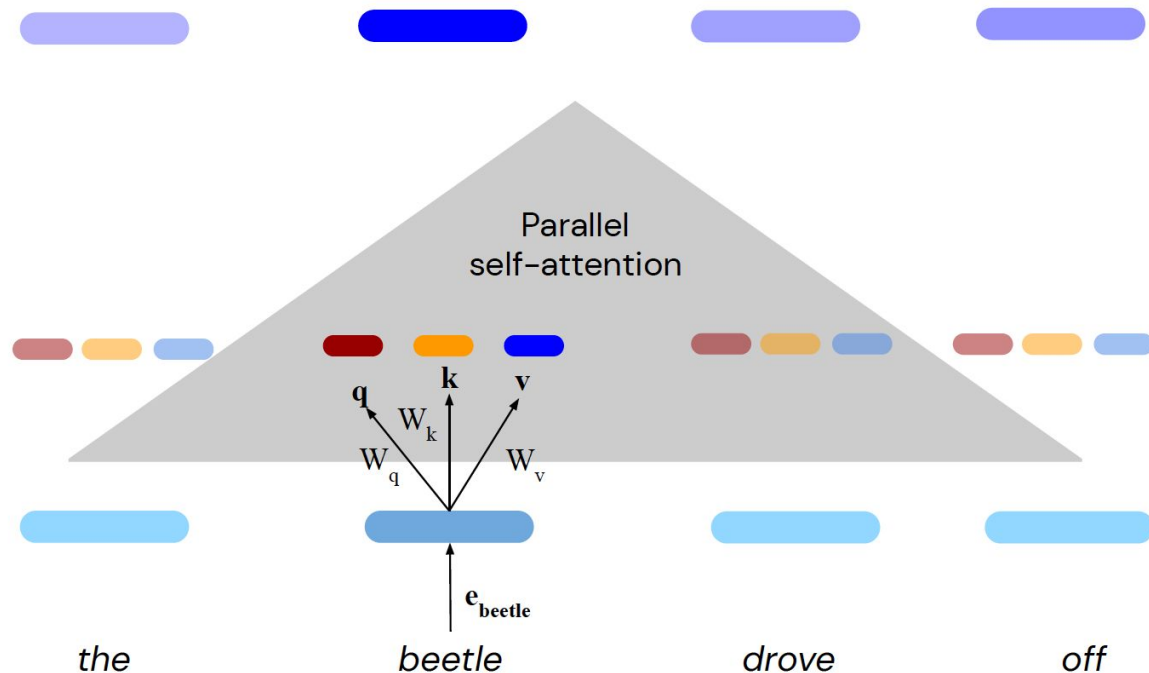


Self-attention over all words (in parallel)



Source: <https://deepmind.com/learning-resources/deep-learning-lecture-series-2020>

Self-attention over all words (in parallel)



Source: <https://deepmind.com/learning-resources/deep-learning-lecture-series-2020>

Self-attention: In equations



The most important formula in deep learning after 2018

Self-Attention

What is self-attention? Self-attention calculates a weighted average of feature representations with the weight proportional to a similarity score between pairs of representations. Formally, an input sequence of n tokens of dimensions d , $X \in \mathbf{R}^{n \times d}$, is projected using three matrices $W_Q \in \mathbf{R}^{d \times d_q}$, $W_K \in \mathbf{R}^{d \times d_k}$, and $W_V \in \mathbf{R}^{d \times d_v}$ to extract feature representations Q , K , and V , referred to as query, key, and value respectively with $d_k = d_q$. The outputs Q , K , V are computed as

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V. \quad (1)$$

So, self-attention can be written as,

$$S = D(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_q}} \right) V, \quad (2)$$

where softmax denotes a *row-wise* softmax normalization function. Thus, each element in S depends on all other elements in the same row.

7:38 AM · Feb 10, 2021



3.1K



Reply



Copy link

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_j = \mathbf{x}_j \mathbf{W}^K; \quad \mathbf{v}_j = \mathbf{x}_j \mathbf{W}^V$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

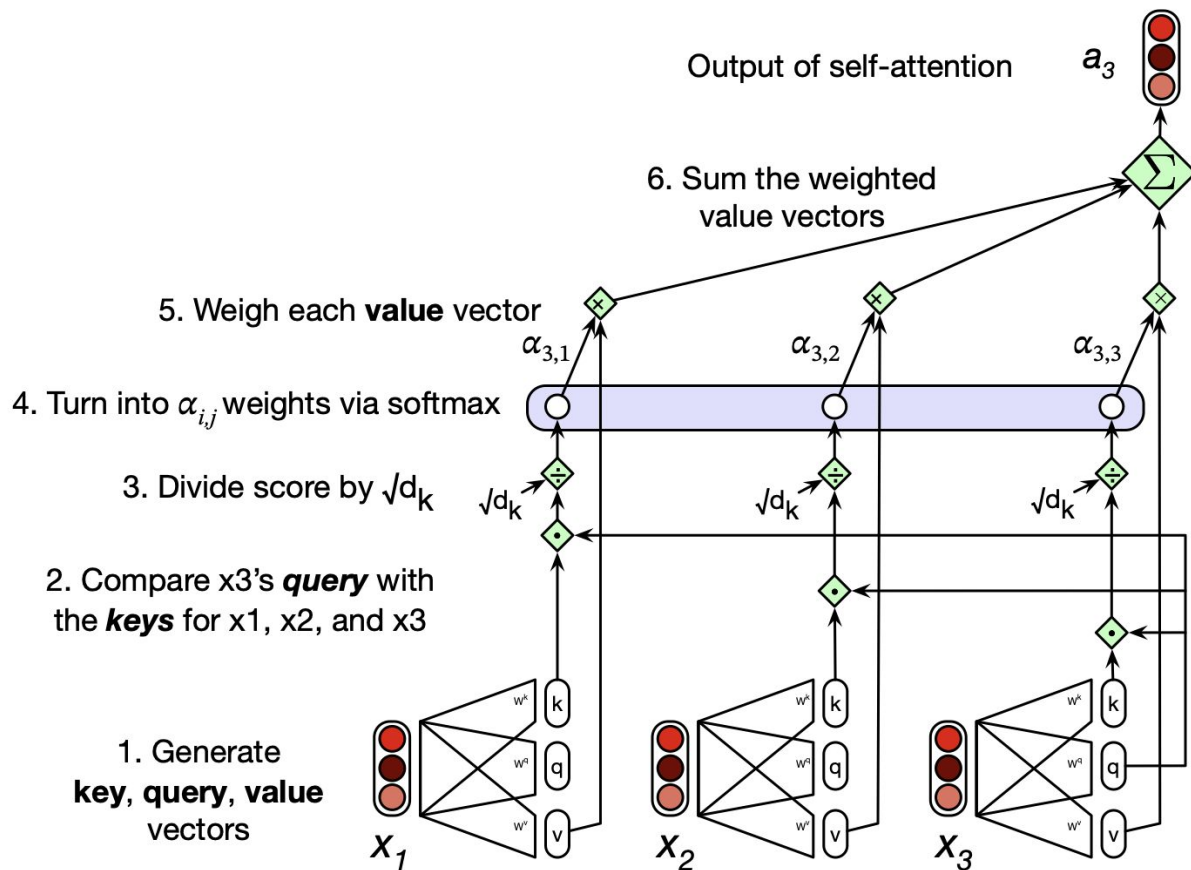
$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j))$$

$$\mathbf{a}_i = \sum \alpha_{ij} \mathbf{v}_j$$

Scaled dot-product: *more on this later*

Source: <https://theaisummer.com/self-attention/> <https://web.stanford.edu/~jura/sky/slp3/>

Calculating the self-attention output



Try this problem



Suppose, you give the following input to your transformer encoder: {flying, arrows} The input embeddings for these two words are **[0,1,1,1,1,0]** and **[1,1,0,-1,-1,1]**, respectively. Suppose you are trying to represent the first word 'flying' with the help of self-attention in the first encoder. For the first attention head, the query, key and value matrices just take the 2 dimensions from the input each. Thus, the first 2 dimensions define the query vector, and so on. What will be the self-attention output for the word 'flying' corresponding to this attention head. You are using the scaled dot vector.

$$\frac{\text{Scaled dot product } q \cdot k}{\sqrt{d_k}}$$

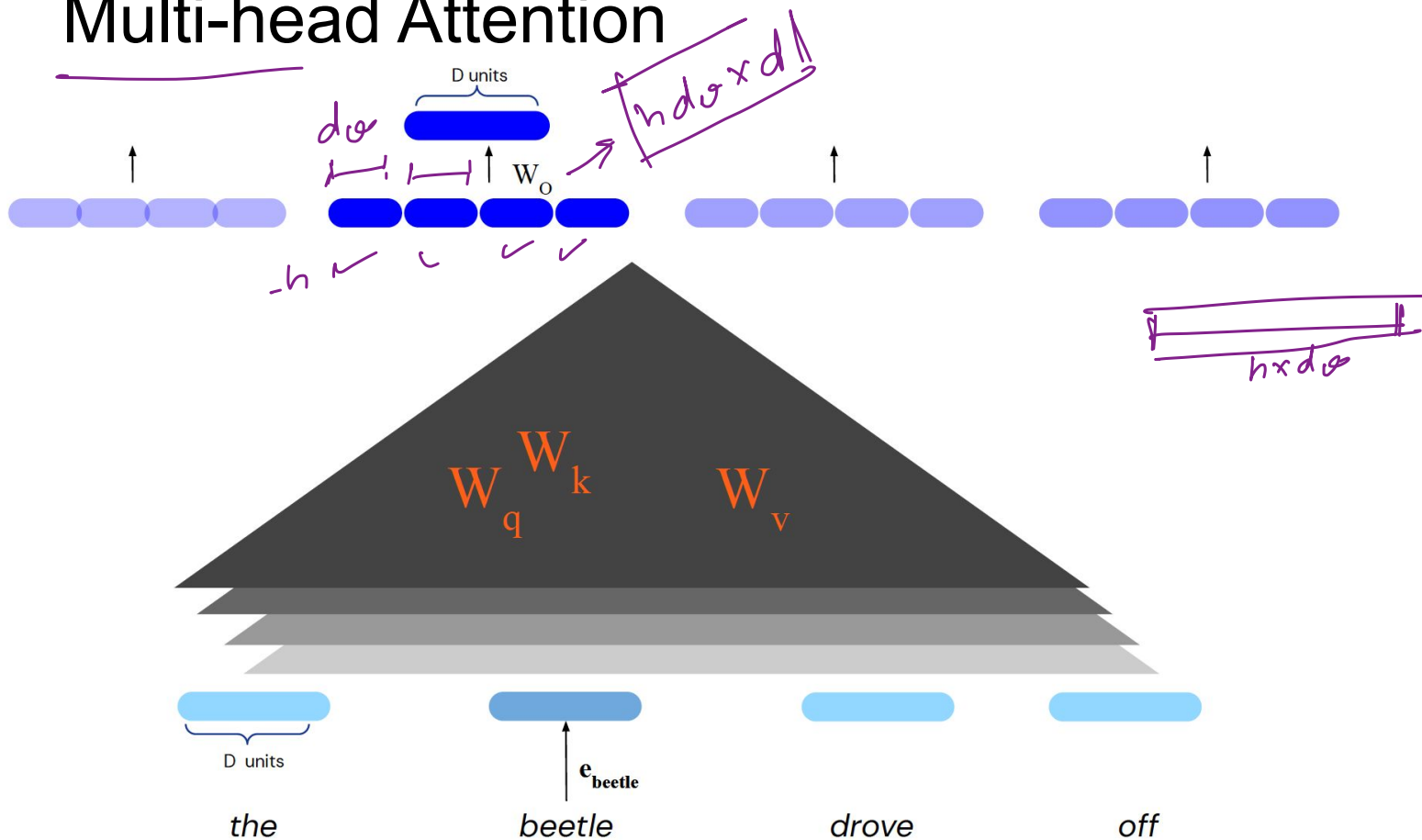
Try this problem

q1: [0,1], k1: [1,1], v1: [1,0]

q2: [1,1], k2: [0,-1], v2: [-1,1]

a1?

Multi-head Attention



Why Multi-head attention?

- What if we want to look in multiple places in the sentence at once?
 - For word i , maybe we want to focus on different j for different reasons?
- We'll define multiple attention “heads” through multiple Q,K,V matrices
- Each attention head performs attention independently
- Then the outputs of all the heads are combined!
- Each head gets to “look” at different things, and construct value vectors differently.

Why Multi-head attention?

Prior work identified three important types of heads by looking at attention matrices

[[Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned](https://arxiv.org/abs/1905.09418): <https://arxiv.org/abs/1905.09418>]

1. **Positional** heads that attend mostly to their neighbor.
2. **Syntactic** heads that point to tokens with a specific syntactic relation.
3. Heads that point to **rare words** in the sentence.

Multi-Head Attention: In Equations

- Each head might be attending to the context for different purposes
 - Different linguistic relationships or patterns in the context

$$\mathbf{q}_i^c = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}^c}; \quad \mathbf{k}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{K}^c}; \quad \mathbf{v}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{V}^c}; \quad \forall c \quad 1 \leq c \leq h$$

$$\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^c \cdot \mathbf{k}_j^c}{\sqrt{d_k}}$$

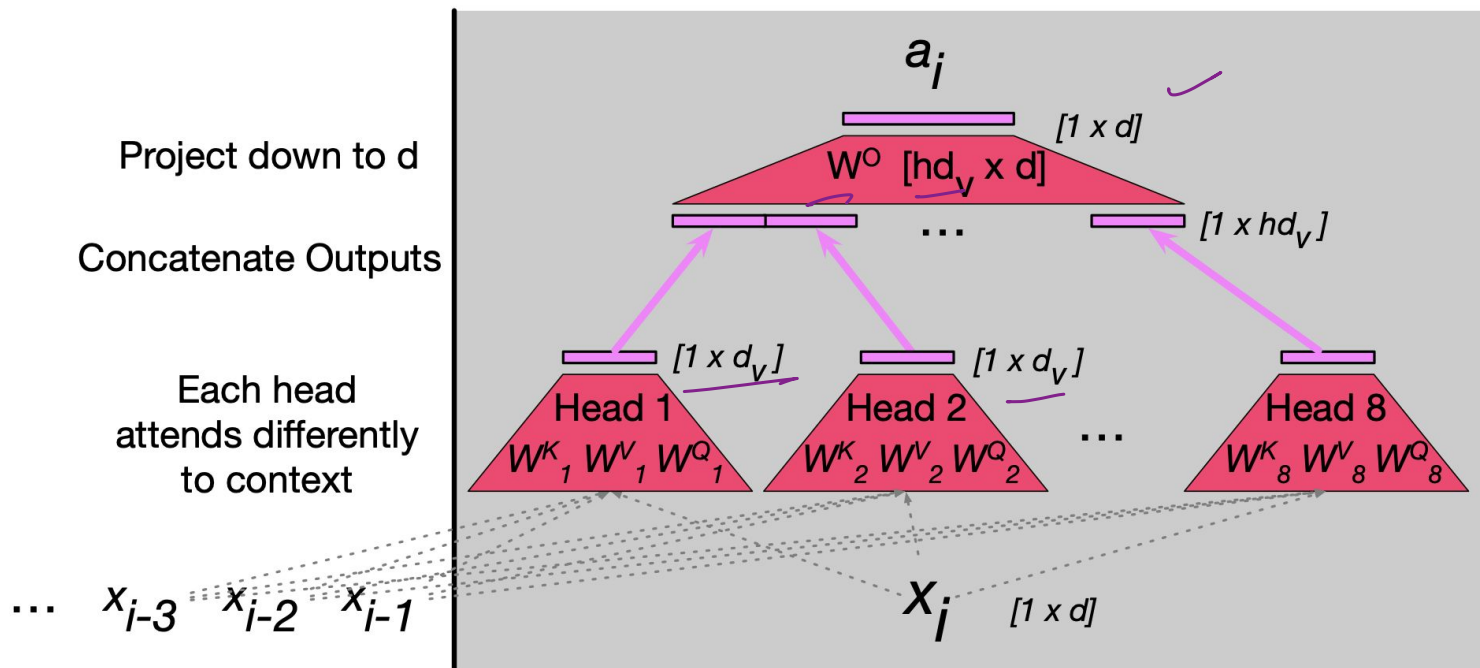
$$\alpha_{ij}^c = \text{softmax}(\text{score}^c(\mathbf{x}_i, \mathbf{x}_j))$$

$$\text{head}_i^c = \sum \alpha_{ij}^c \mathbf{v}_j^c$$

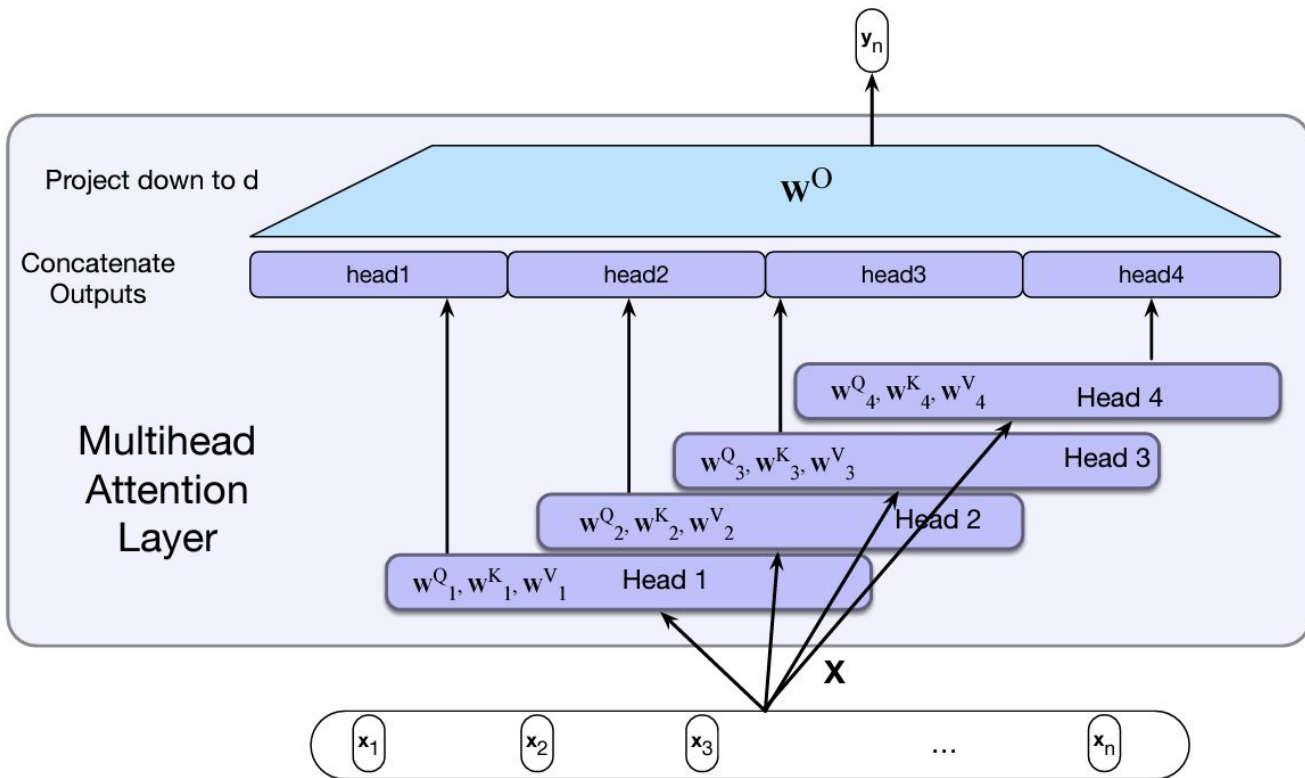
$$\mathbf{a}_i = (\text{head}^1 \oplus \text{head}^2 \dots \oplus \text{head}^h) \mathbf{W}^O$$

$$\text{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \dots, \mathbf{x}_N]) = \mathbf{a}_i$$

Multi-head attention



Multi-head Attention Layer



More on dimensions

Model dimension: d ($=512$)

Query, Key, Value dimensions:

d_q, d_k, d_v ($=64$ each)

Projection matrices: $d \times d_k$ ($= 512 \times 64$ each)

The output at each head: d_v

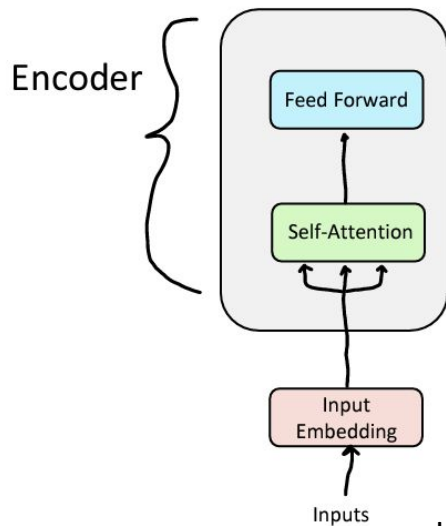
For " h " ($=8$) multi-heads: hd_v

To project it back to model dimension: $w^O: d \times hd_v$

Feed-forward Layer

Problem: Since there are no element-wise non-linearities, self-attention is simply performing a re-averaging of the value vectors.

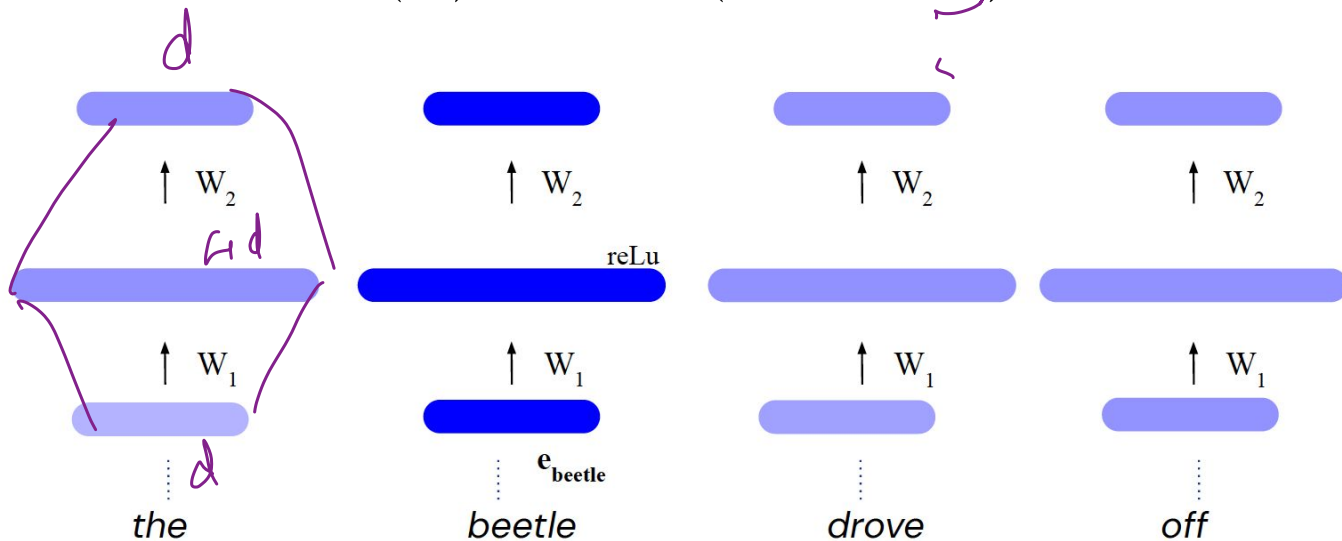
Easy fix: Apply a feedforward layer to the output of attention, providing non-linear activation (and additional expressive power).



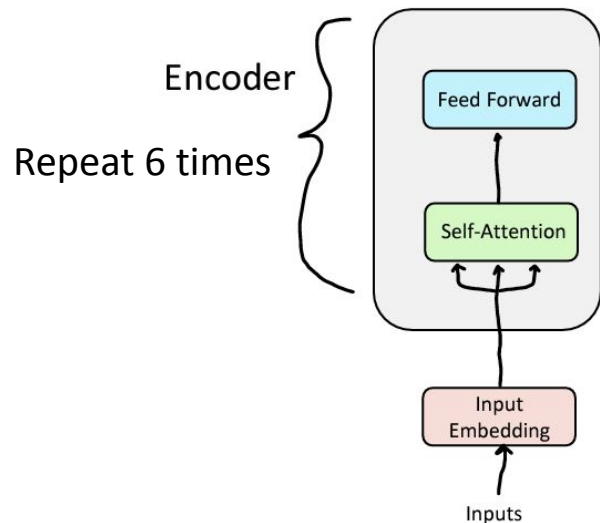
Feed-forward Layer

- ① residual connection
- ② Layer Norm.
- ③ Scaled dot product

$$\text{FFN}(\mathbf{x}_i) = \text{ReLU}(\mathbf{x}_i \mathbf{W}_1 + b_1) \mathbf{W}_2 + b_2$$



How to make this work for deep networks?



Training Trick #1: Residual Connections

Training Trick #2: LayerNorm

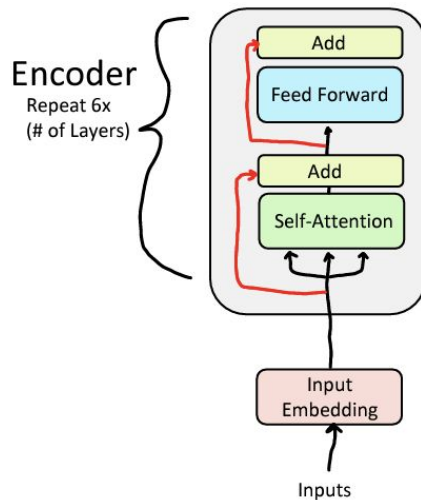
Training Trick #3: Scaled Dot Product Attention

Training Trick #1: Residual Connections

- Residual connections are a simple but powerful technique from computer vision.
- Deep networks are surprisingly bad at learning the identity function!
- Therefore, directly passing "raw" embeddings to the next layer can actually be very helpful!

$$x_{\ell} = F(x_{\ell-1}) + x_{\ell-1}$$

- This prevents the network from "forgetting" or distorting important information as it is processed by many layers.

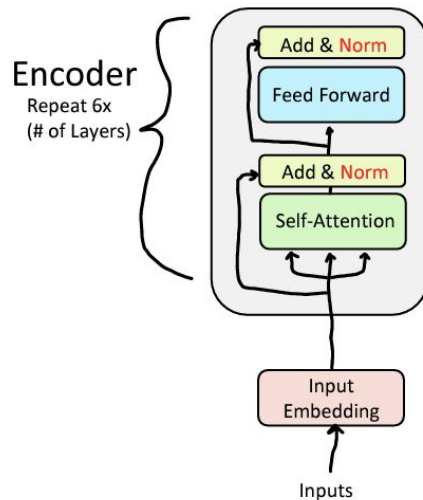


Training Trick #2: Layer Normalization

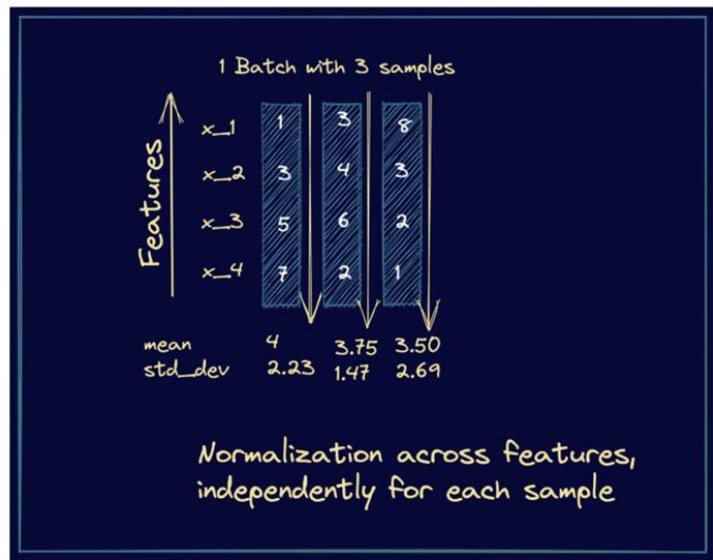
- **Problem:** Difficult to train the parameters of a given layer because its input from the layer beneath keeps shifting.
- **Solution:** Reduce variation by **normalizing** to zero mean and standard deviation of one within each **layer**.

$$\text{Mean: } \mu^l = \frac{1}{H} \sum_{i=1}^H a_i^l \quad \text{Standard Deviation: } \sigma^l = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_i^l - \mu^l)^2}$$

$$x^{\ell'} = \frac{x^{\ell} - \mu^{\ell}}{\sigma^{\ell} + \epsilon}$$



Training Trick #2: Layer Normalization

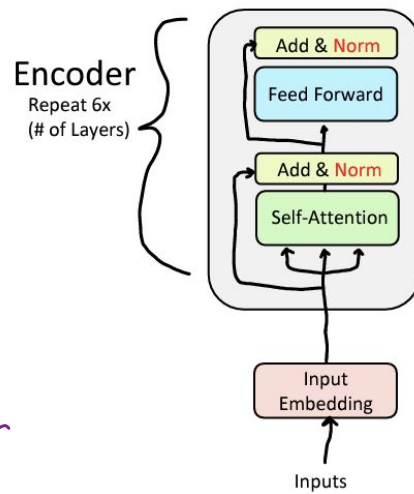


An Example of How LayerNorm Works (Image by Bala Priya C, Pinecone)

Mean: $\mu^l = \frac{1}{H} \sum_{i=1}^H a_i^l$ Standard Deviation: $\sigma^l = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_i^l - \mu^l)^2}$

$$x^{\ell'} = \frac{x^{\ell} - \mu^{\ell}}{\sigma^{\ell} + \epsilon}$$

<https://web.stanford.edu/class/cs224n/>

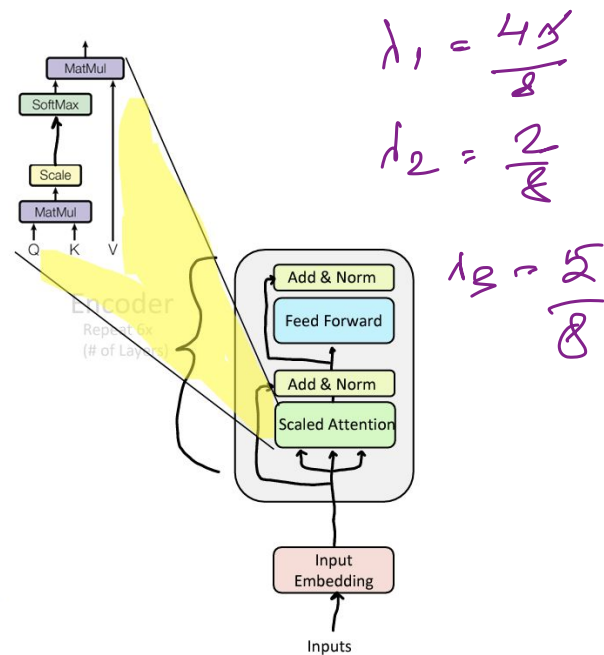


Training Trick #3: Scaled Dot Product Attention

- After LayerNorm, the mean and variance of vector elements is 0 and 1, respectively. (Yay!)
- However, the dot product still tends to take on extreme values, as its variance scales with dimensionality d_k

Quick Statistics Review:

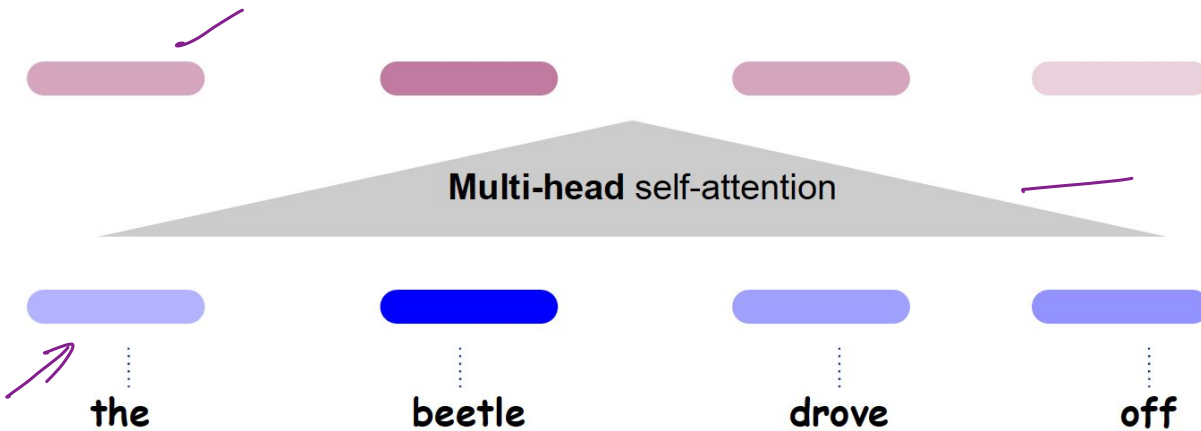
- Mean of sum = sum of means = $d_k * 0 = 0$
- Variance of sum = sum of variances = $d_k * 1 = d_k$
- To set the variance to 1, simply divide by $\sqrt{d_k}$!



Training Trick #3: Scaled Dot Product Attention

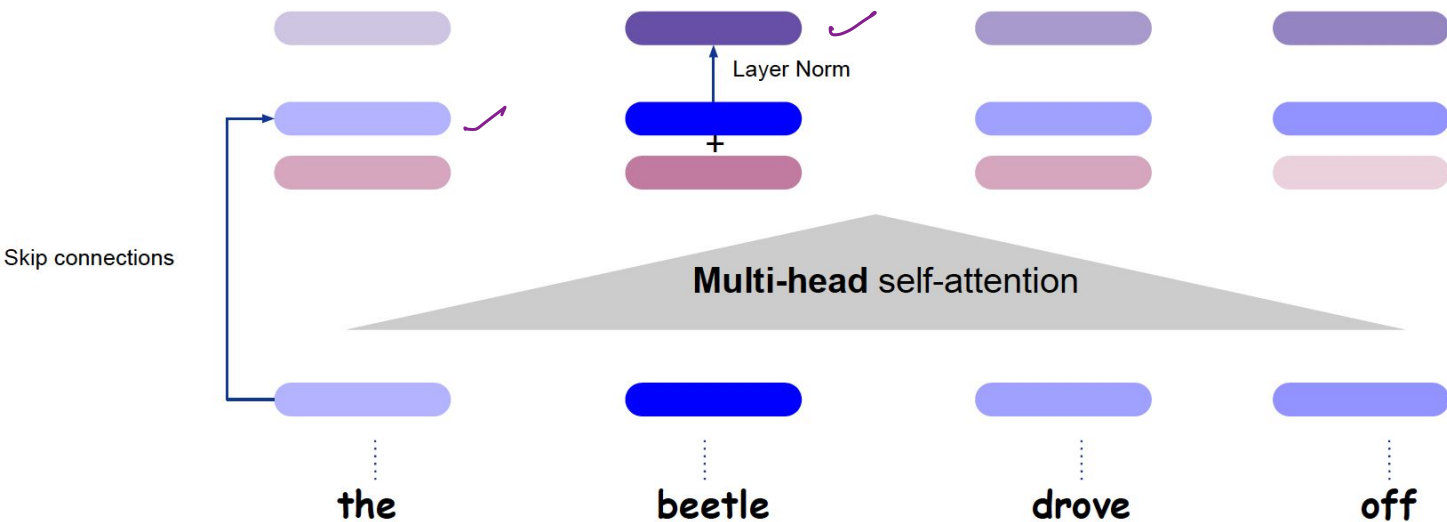
- Assume that the components of q and k are independent random variables with mean 0 and variance 1.
- Then their dot product $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ has mean 0 and variance d_k .
- Hence the scaling ensures that the resultant dot product has mean 0 and variance 1.

A Complete Transformer Block



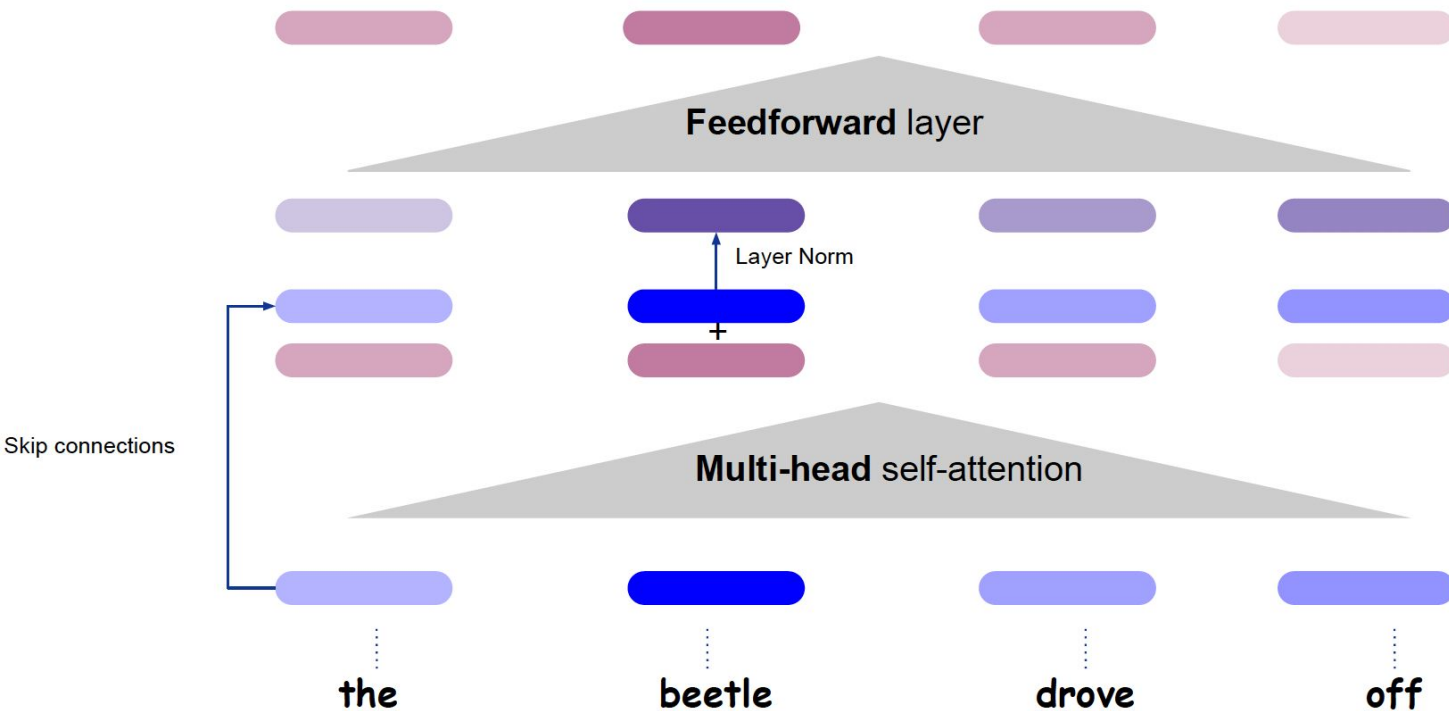
Source: <https://deepmind.com/learning-resources/deep-learning-lecture-series-2020>

A Complete Transformer Block



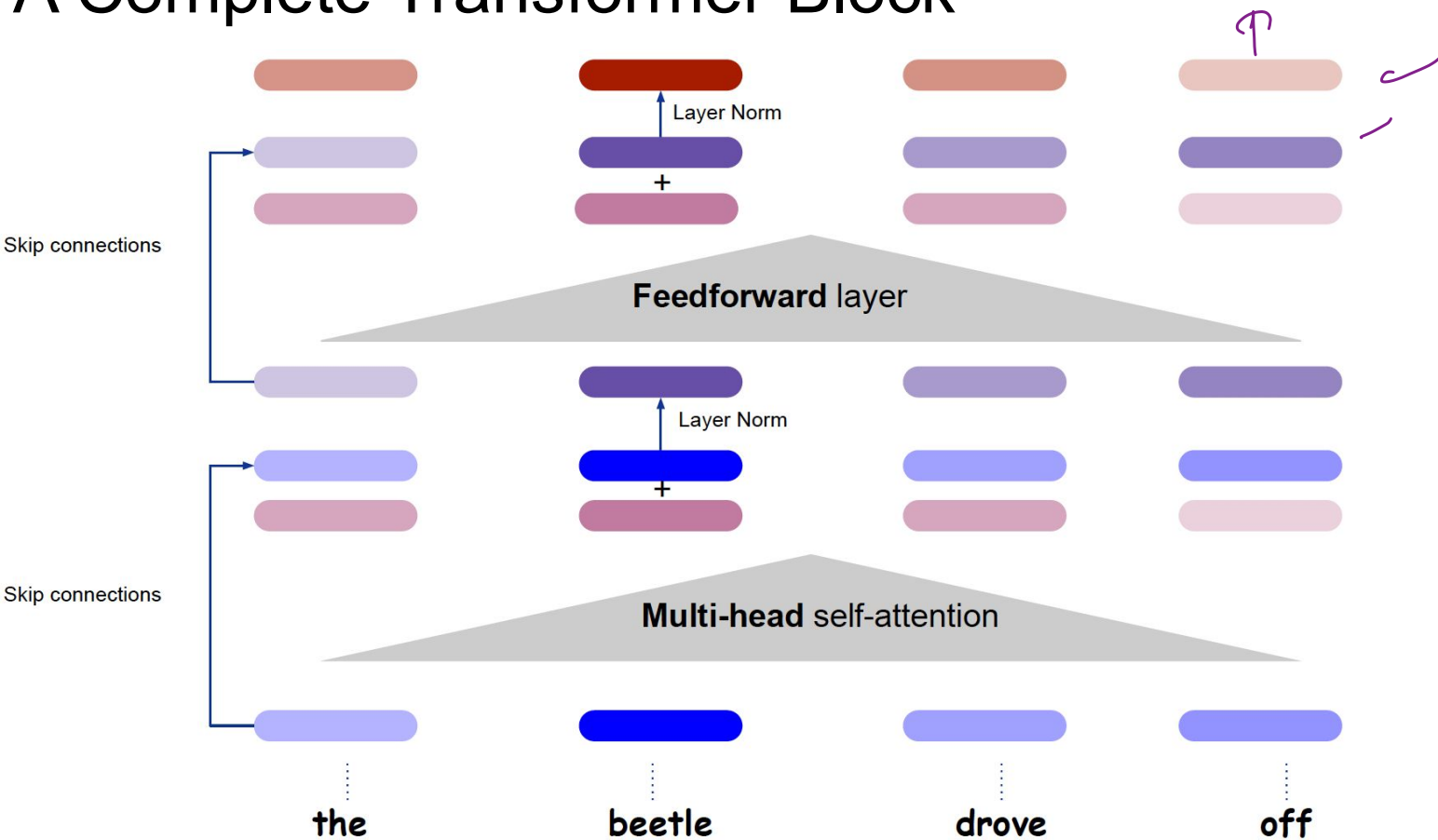
Source: <https://deepmind.com/learning-resources/deep-learning-lecture-series-2020>

A Complete Transformer Block



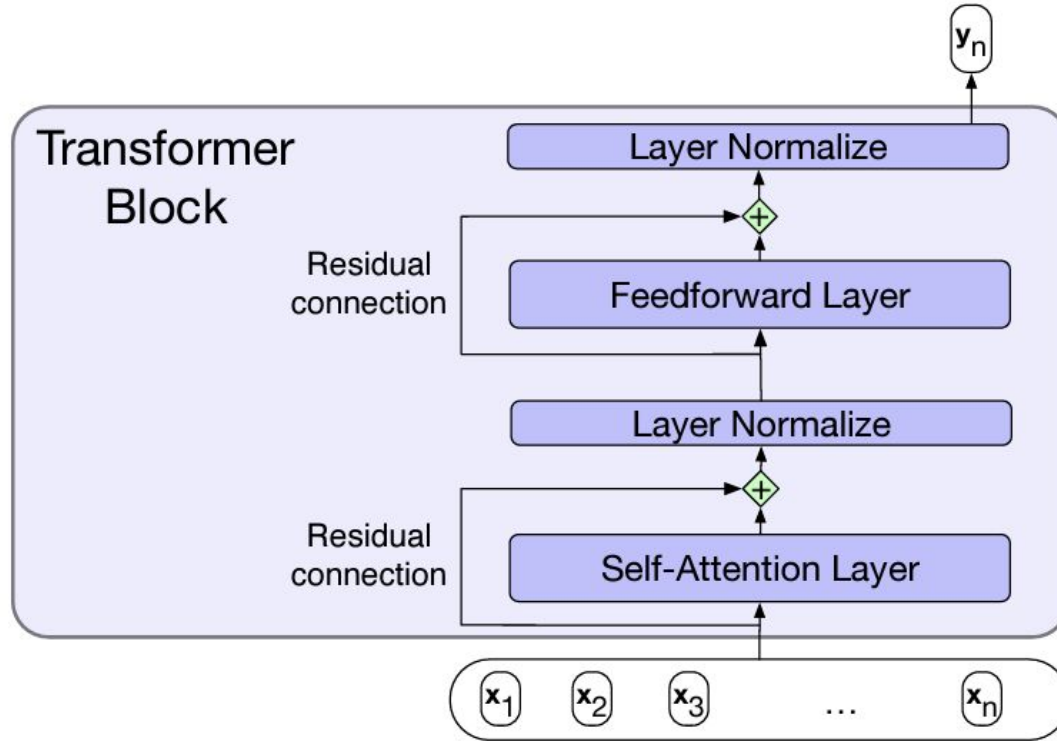
Source: <https://deepmind.com/learning-resources/deep-learning-lecture-series-2020>

A Complete Transformer Block



Source: <https://deepmind.com/learning-resources/deep-learning-lecture-series-2020>

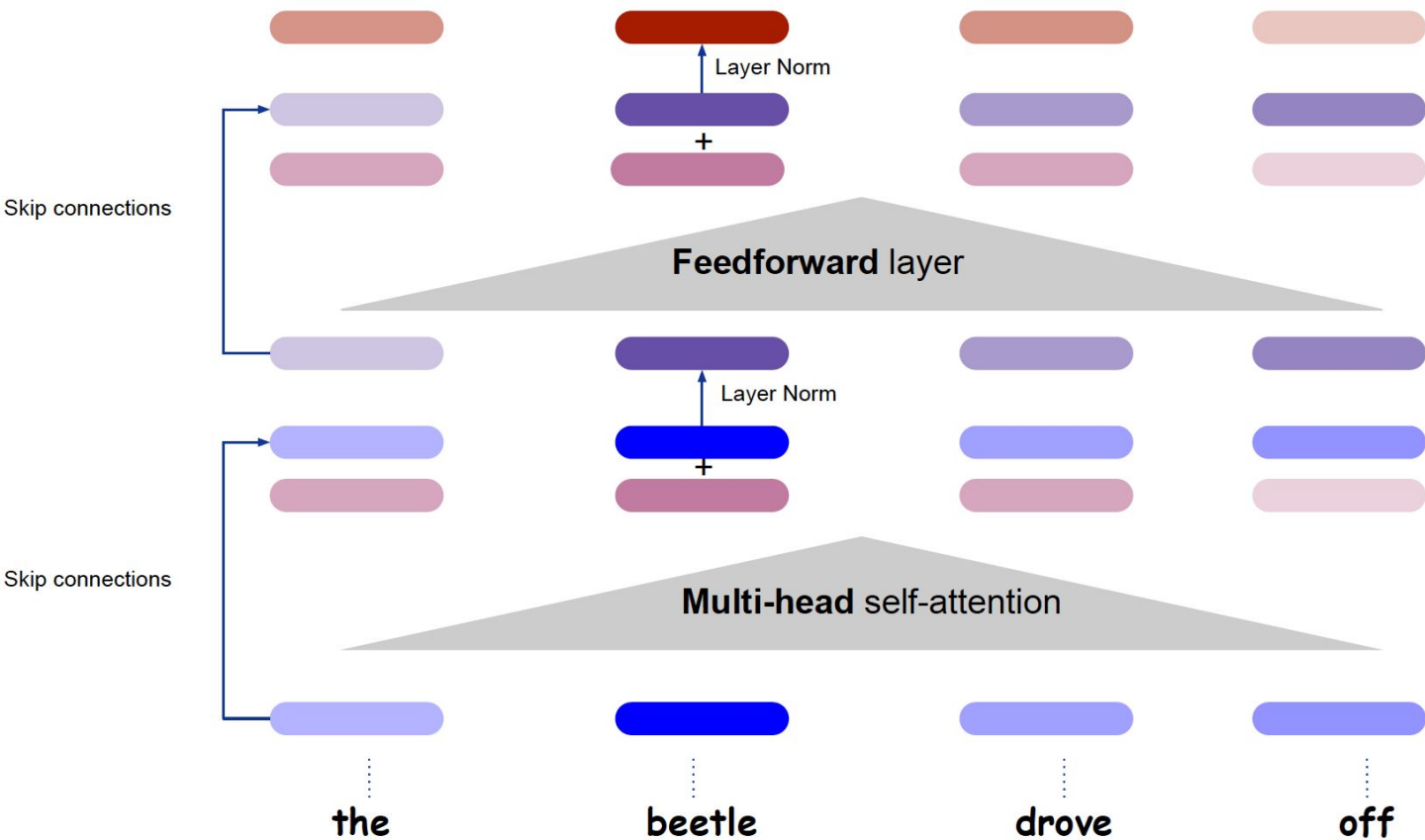
Transformer Block: Another visualization



$$\mathbf{z} = \text{LayerNorm}(\mathbf{x} + \text{SelfAttention}(\mathbf{x}))$$

$$\mathbf{y} = \text{LayerNorm}(\mathbf{z} + \text{FFN}(\mathbf{z}))$$

A Complete Transformer Block

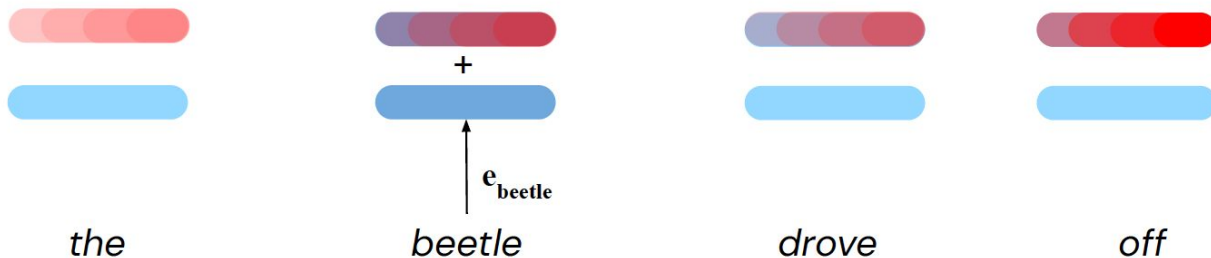


There is still a major problem!

Order does not matter!!

Position Encoding of words

- Add fixed quantity to embedding activations
- The quantity added to each input embedding unit $\in [-1, 1]$ depends on:
 - The dimension of the unit within the embedding
 - The (absolute) position of the words in the input



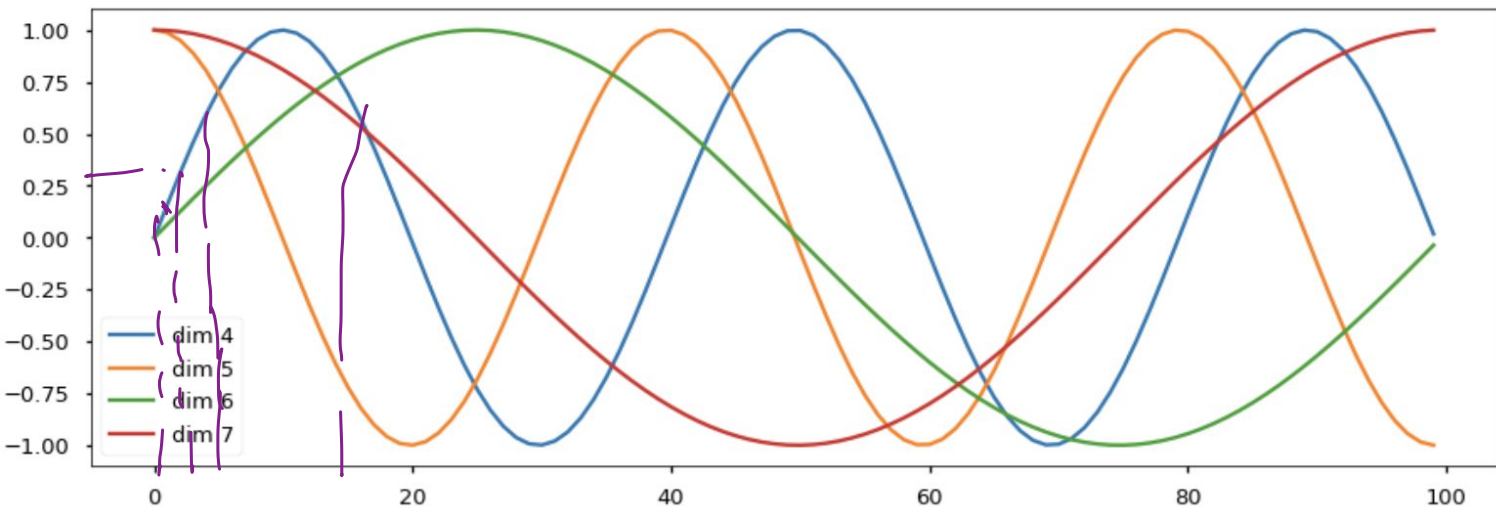
Source: <https://deepmind.com/learning-resources/deep-learning-lecture-series-2020>

Understanding Positional Encoding

Sinusoidal position representations: concatenate
sinusoidal functions of varying periods

$p_i =$

$$\begin{pmatrix} \sin(i/10000^{2*1/d}) \\ \cos(i/10000^{2*1/d}) \\ \vdots \\ \sin(i/10000^{2*\frac{d}{2}/d}) \\ \cos(i/10000^{2*\frac{d}{2}/d}) \end{pmatrix}$$



Understanding Positional Encoding

Suppose you have an input sequence of length L

Defining the positional encoding of the k^{th} word within the sequence. The positional encoding is given by **sine** and **cosine** functions of varying frequencies:

$$\text{even p.e.} \quad P(k, 2i) = \sin\left(\frac{k}{n^{2i/d}}\right)$$

$$\text{odd p.e.} \quad P(k, 2i + 1) = \cos\left(\frac{k}{n^{2i/d}}\right)$$

Here:

k : Position of a word in the input sequence

i : Used for mapping to indices in the positional encoding of a particular word $0 \leq i < d/2$

d : Dimension of the output embedding space

$P(k, j)$: Position function for mapping a position k in the input sequence to index (k, j) of the positional matrix

n : User-defined scalar, set to 10,000 by the authors of [Attention Is All You Need](https://arxiv.org/abs/2018.08.08).

Understanding Positional Encoding

Sequence

Index of token, k

Positional Encoding Matrix with $d=4$, $n=100$

		$i=0$	$i=0$	$i=1$	$i=1$
I	0	$P_{00}=\sin(0)$ = 0	$P_{01}=\cos(0)$ = 1	$P_{02}=\sin(0)$ = 0	$P_{03}=\cos(0)$ = 1
am	1	$P_{10}=\sin(1/1)$ = 0.84	$P_{11}=\cos(1/1)$ = 0.54	$P_{12}=\sin(1/10)$ = 0.10	$P_{13}=\cos(1/10)$ = 1.0
a	2	$P_{20}=\sin(2/1)$ = 0.91	$P_{21}=\cos(2/1)$ = -0.42	$P_{22}=\sin(2/10)$ = 0.20	$P_{23}=\cos(2/10)$ = 0.98
Robot	3	$P_{30}=\sin(3/1)$ = 0.14	$P_{31}=\cos(3/1)$ = -0.99	$P_{32}=\sin(3/10)$ = 0.30	$P_{33}=\cos(3/10)$ = 0.96

$$P(k, 2i) = \sin\left(\frac{k}{n^{2i/d}}\right)$$

$$P(k, 2i + 1) = \cos\left(\frac{k}{n^{2i/d}}\right)$$

Other variants: Learned Positional embedding

Goal: learn a position embedding matrix E_{pos} of shape $[1 \times N]$.

Start with randomly initialized embeddings

- one for each integer up to some maximum length.
- i.e., just as we have an embedding for token *fish*, we'll have an embedding for position 3 and position 17.
- As with word embeddings, these position embeddings are learned along with other parameters during training.

Multi-headed Self Attention Block

1) This is our input sentence*

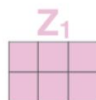
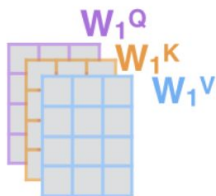
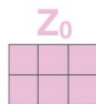
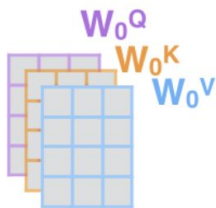
2) We embed each word*

3) Split into 8 heads.
We multiply X or R with weight matrices

4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

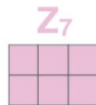
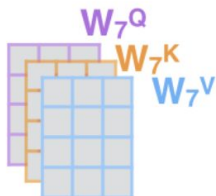
Thinking
Machines



...

...

...



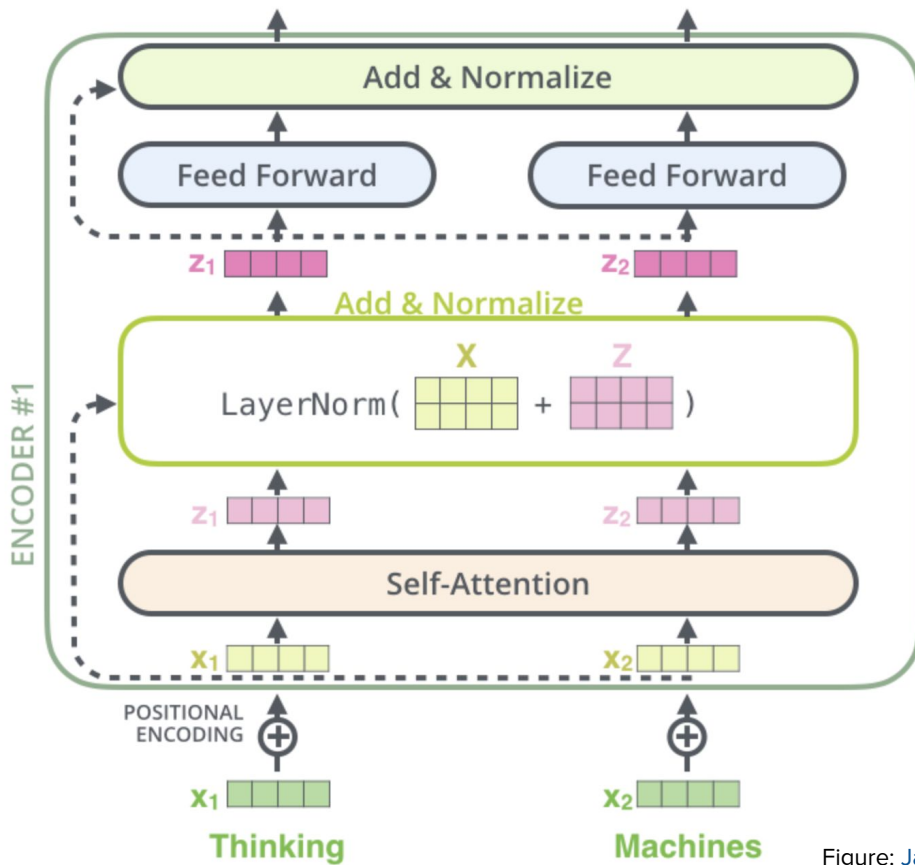
W^O



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

Figure: [Jay Alammar](#)

Encoder Block



$$\frac{3m}{d} = 5/2$$

Figure: [Jay Alammar](#)

Transformer with encoders and decoders

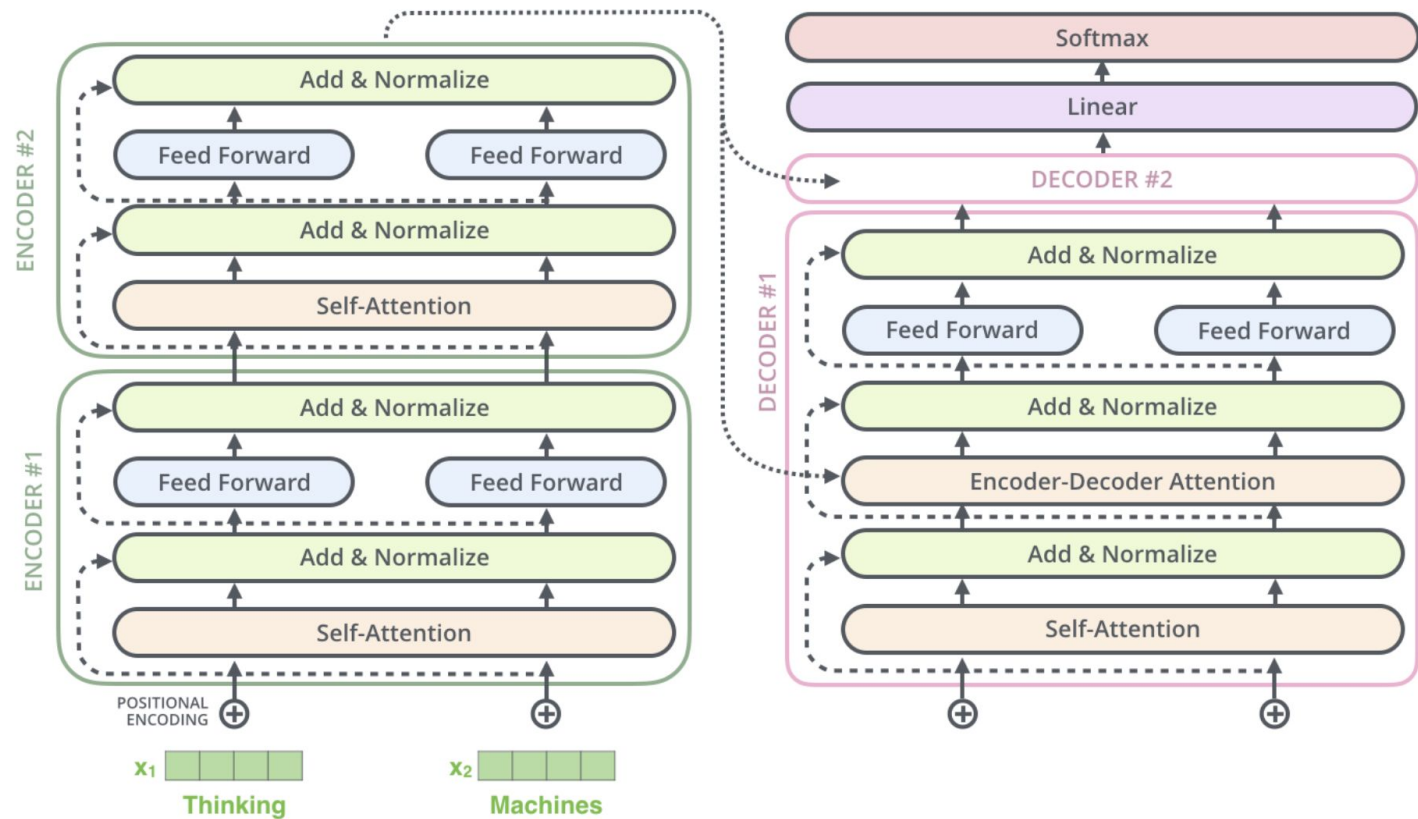
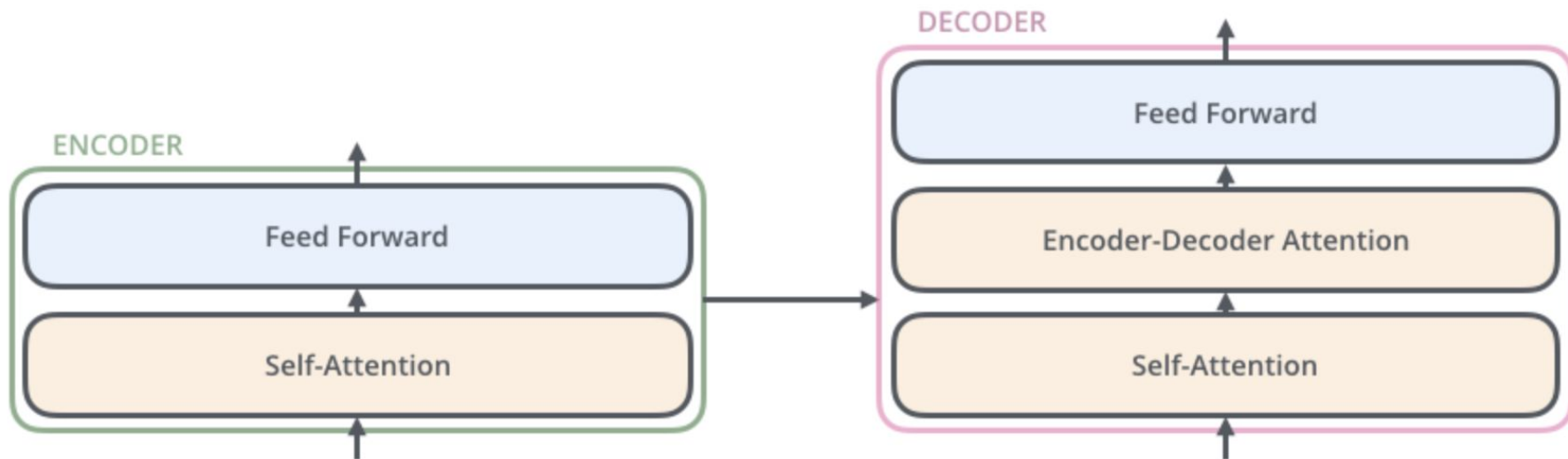


Figure: [Jay Alammar](#)

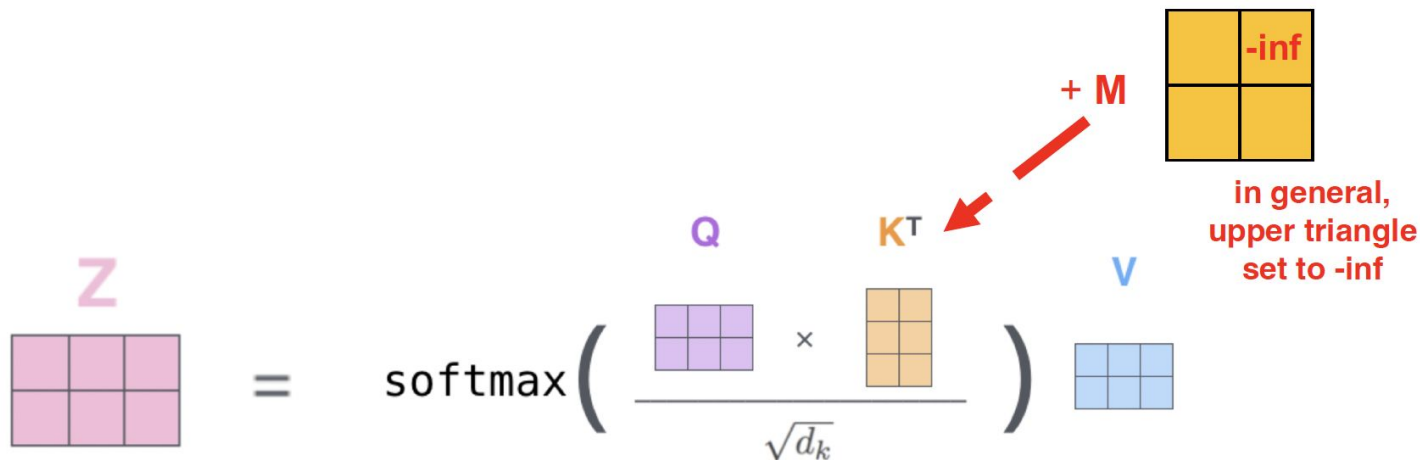
How is the decoder different?



Masked Self-attention

In the **decoder**, the self-attention layer is only allowed to attend to **earlier positions** in the output sequence. Otherwise, we'd be cheating!

This is done by **masking future positions** (setting the dot product score for **future positions to -inf**) before the Softmax step in the self-attention calculation.



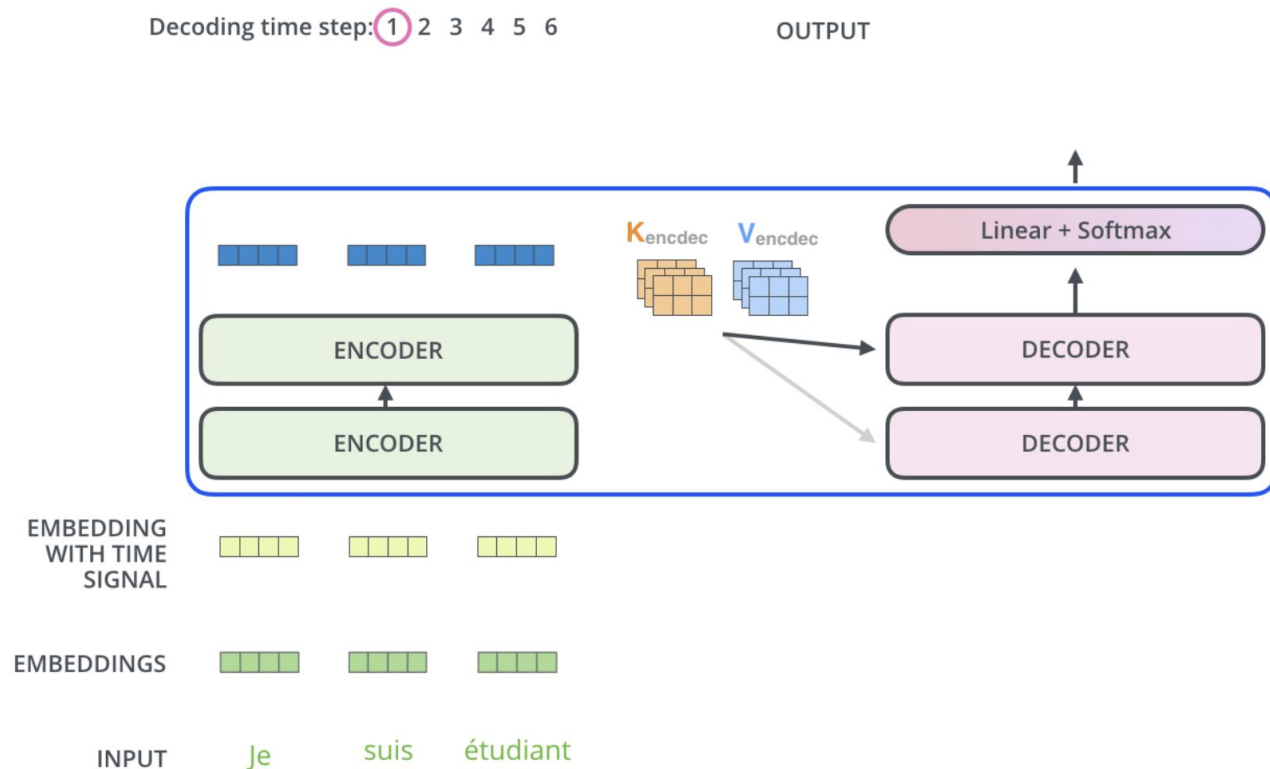
Masked Self-attention for decoder (to avoid seeing the future tokens)

N

q1•k1	$-\infty$	$-\infty$	$-\infty$	$-\infty$
q2•k1	q2•k2	$-\infty$	$-\infty$	$-\infty$
q3•k1	q3•k2	q3•k3	$-\infty$	$-\infty$
q4•k1	q4•k2	q4•k3	q4•k4	$-\infty$
q5•k1	q5•k2	q5•k3	q5•k4	q5•k5

N

Encoder-Decoder Attention



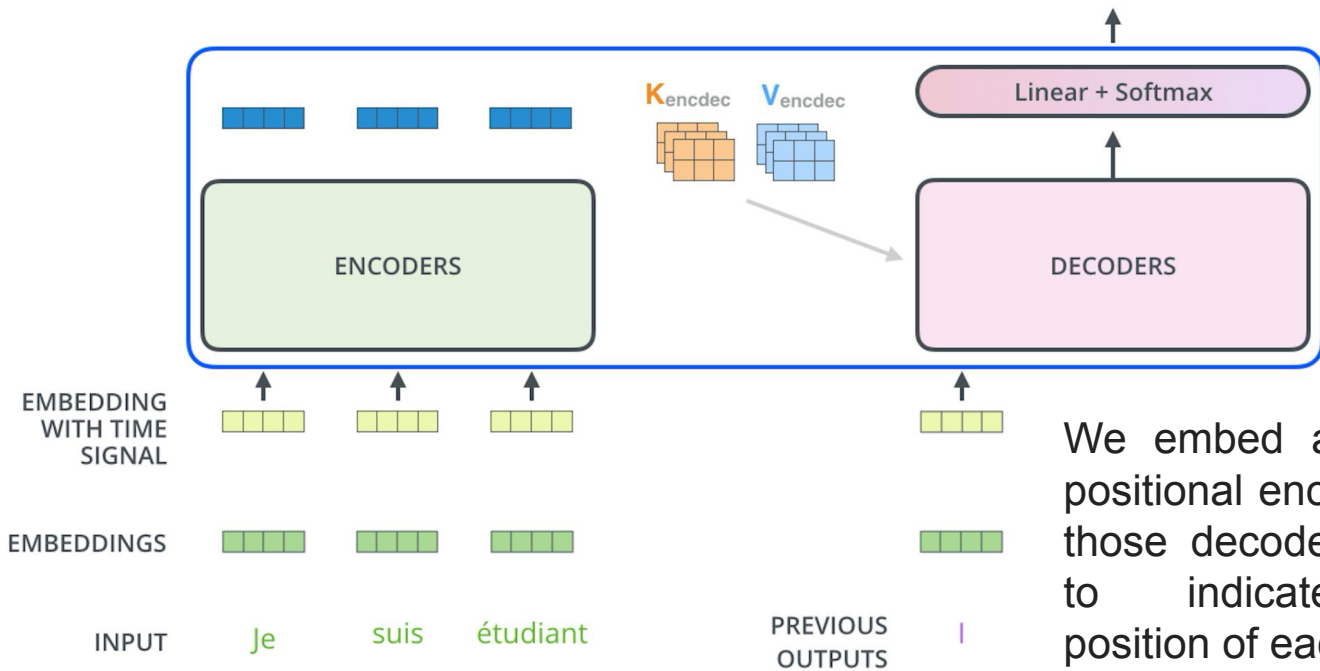
The output of the top encoder is transformed into a set of attention vectors K and V .

These are to be used by each decoder in its “**encoder-decoder attention**” layer which helps the decoder focus on appropriate places in the input sequence:

Decoding

Decoding time step: 1 2 3 4 5 6

OUTPUT | am



The output of each step is fed to the bottom decoder in the next time step, and the decoders bubble up their decoding results just like the encoders did.

We embed and add positional encoding to those decoder inputs to indicate the position of each word.

Converting decoder stack output to words

Which word in our vocabulary
is associated with this index?

Get the index of the cell
with the highest value
(argmax)

am

5

That's the job of the final
Linear layer which is
followed by a **Softmax**
Layer.

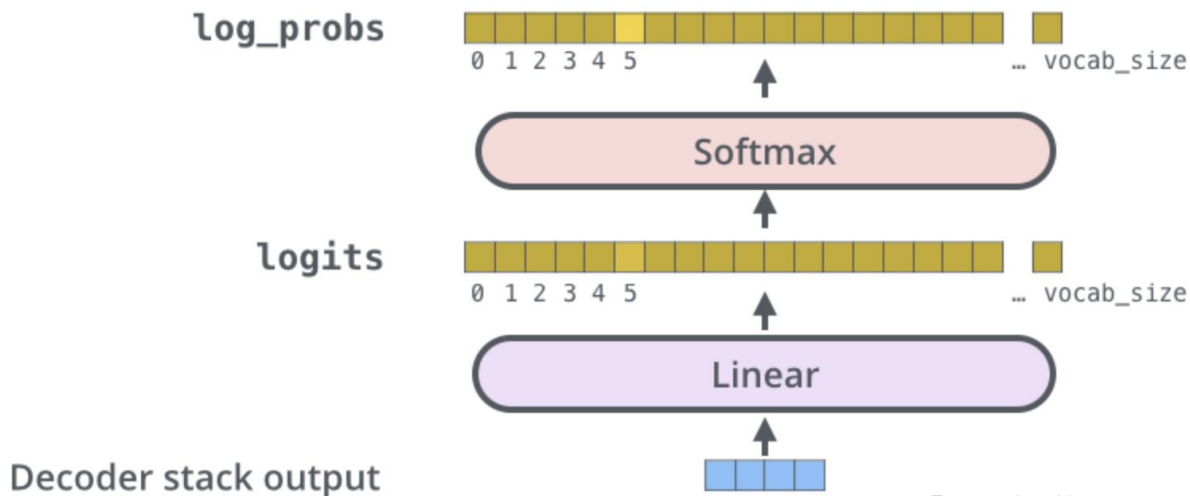
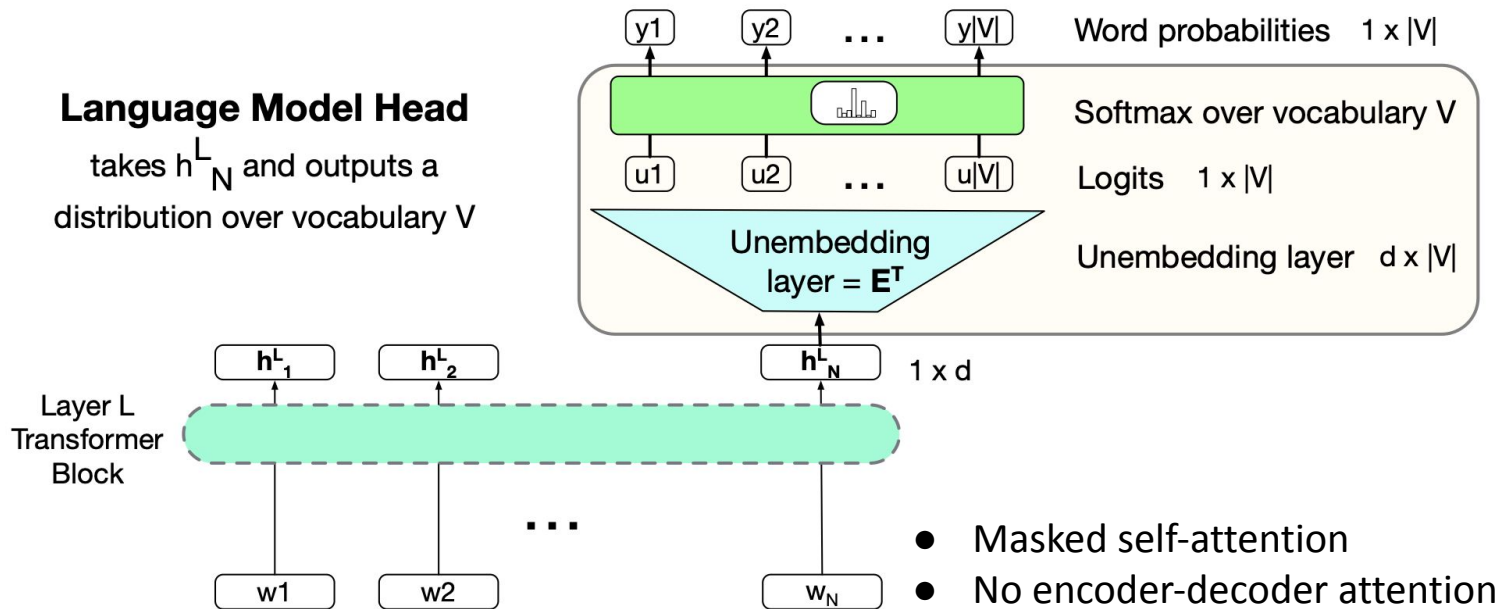


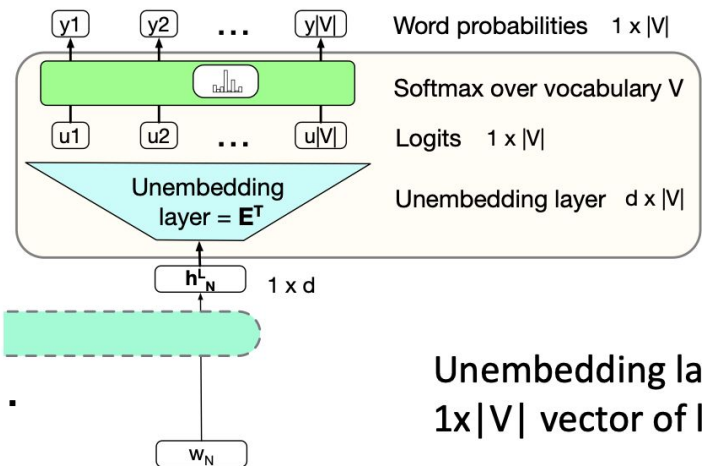
Figure: [Jay Alamar](#)

Transformers as Language models



Transformers as Language models

Unembedding layer: linear layer projects from h_N^L (shape $[1 \times d]$) to logit vector



Why "unembedding"? **Tied to E^T**

Weight tying, we use the same weights for two different matrices

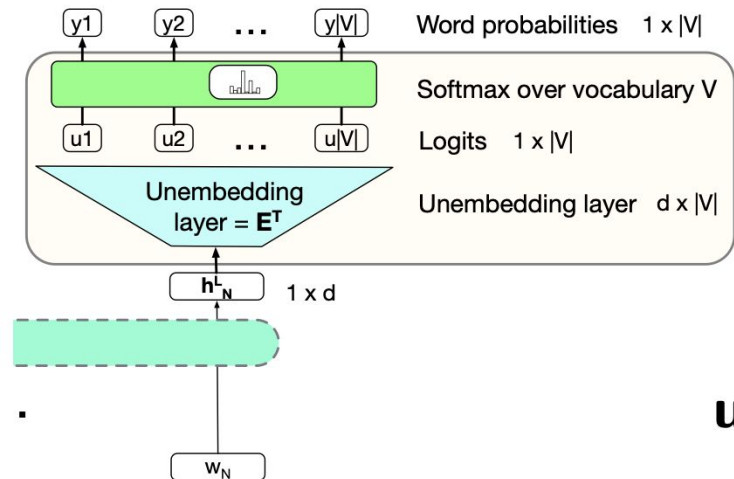
Unembedding layer maps from an embedding to a $1 \times |V|$ vector of logits

Language modeling head

Logits, the score vector $\mathbf{u}$

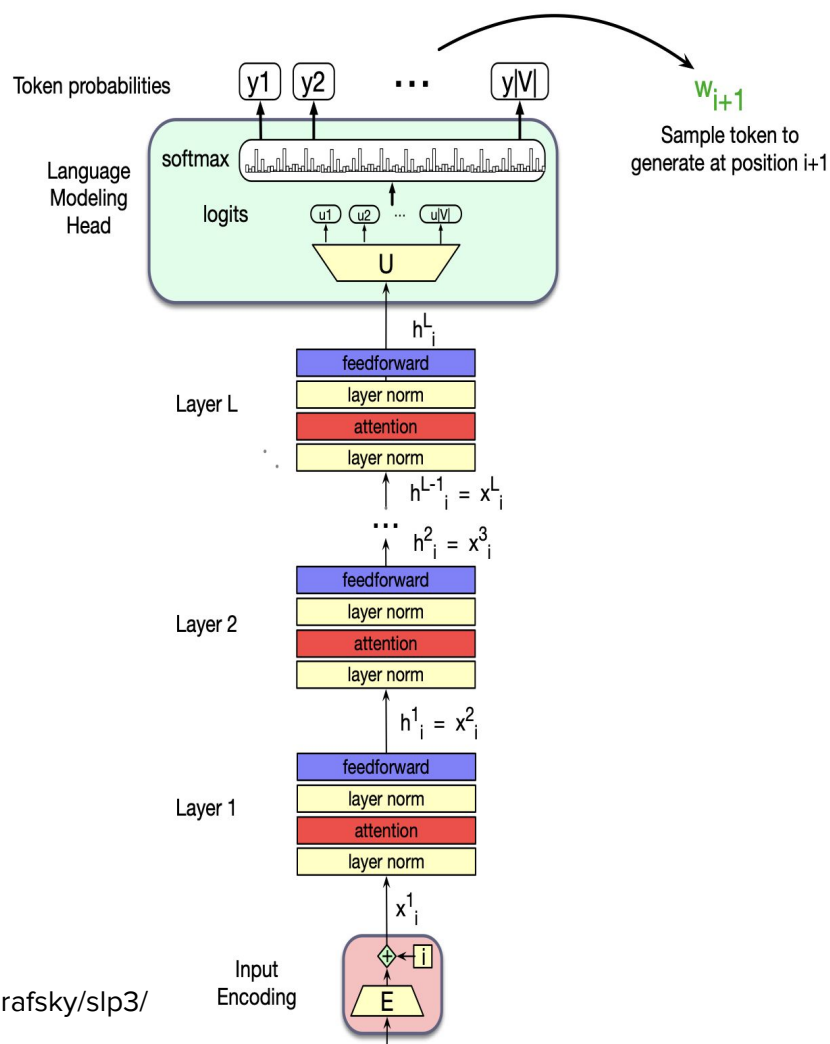
One score for each of the $|V|$ possible words in the vocabulary V .
Shape $1 \times |V|$.

Softmax turns the logits into probabilities over vocabulary.
Shape $1 \times |V|$.



$$\mathbf{u} = \mathbf{h}_N^L \mathbf{E}^T$$
$$\mathbf{y} = \text{softmax}(\mathbf{u})$$

Transformers LM



<https://web.stanford.edu/~jurafsky/slp3/>

Number of parameters in encoder-decoder?

$$\begin{aligned} & \left[\underset{\substack{\uparrow \\ \text{model dim}}}{3 \times d \times \left[\frac{d}{h} \right]} \right] \times h \rightarrow \# \text{ heads} + [h d_v \times d] \\ & \quad \downarrow \\ & \quad \# \text{ heads} \quad \text{query, key, value dim} = 1 \times \frac{d}{h} \\ & \quad + [d \times 4d + 4d \times d] \end{aligned}$$

Number of parameters in the decoder only Transformer?

$$12d^2L$$

$$d = 128$$

$$L = 96$$

$$\frac{\text{GPT-3 parameters}}{175\text{B}}$$